POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Adaptive MPC controller through Deep Reinforcement Learning for Proximity Operations in HEO

Tesi di Laurea Magistrale in
Space Engineering - Ingegneria Spaziale

Author: **Pietro Cavalletti**

Student ID: 235820
Advisor: Prof. Mauro Massari
Co-advisors: Luca Giorcelli
Academic Year: 2024-25

Abstract

Autonomous spacecraft proximity operations in Highly Elliptical Orbits (HEO) are critical for emerging on-orbit servicing and debris removal missions, yet they pose significant guidance and control (G&C) challenges due to complex orbital dynamics and stringent safety constraints. Traditional G&C methods often struggle with the trade-off between performance and the high computational load on resource-constrained flight hardware. This thesis introduces a novel hybrid G&C framework that synergistically integrates Deep Reinforcement Learning (DRL) with Model Predictive Control (MPC) to enhance autonomy and efficiency. The core contribution is a DRL agent trained to dynamically tune key MPC parameters in real-time, specifically the prediction horizon, control timestep, and the state and control weighting matrices. This approach leverages MPC's predictive, constraint-aware nature while using DRL's adaptive learning to mitigate its high computational cost and reliance on manual tuning. The findings, validated through extensive high-fidelity Monte Carlo simulations, demonstrate that the DRL-tuned MPC significantly improves performance over statically configured controllers. The hybrid system achieves substantial optimization in both fuel efficiency and computational performance, with fuel savings ranging from 10% to 40% and a reduction in relative computational time by approximately 65%. Crucially, these gains are obtained without compromising the safety guarantees inherent to the MPC framework; the DRL agent acts as an expert tuner, but the underlying MPC layer remains the final arbiter responsible for enforcing all system dynamics and operational constraints. The developed system also exhibits superior robustness to unmodeled disturbances and parameter uncertainties, maintaining a perfect mission success rate across a wide range of operational conditions. This work provides definitive evidence that this hybrid strategy is a viable, robust, and efficient solution for real-time, on-board implementation in autonomous spacecraft guidance.

Keywords: Deep Reinforcement Learning (DRL) Model Predictive Control (MPC) Spacecraft Proximity Operations Highly Elliptical Orbits (HEO) Autonomous Control Hybrid Control Systems

Abstract in lingua italiana

Le operazioni autonome di prossimità tra veicoli spaziali in Orbite Altamente Ellittiche (HEO) sono cruciali per le nuove missioni di servizio in orbita e di rimozione dei detriti, ma presentano significative sfide di controllo a causa di dinamiche orbitali complesse e stringenti vincoli di sicurezza. I metodi di controllo tradizionali spesso faticano a trovare un compromesso tra prestazioni e l'elevato carico computazionale su hardware di volo con risorse limitate. Questa tesi introduce un nuovo framework di controllo ibrido che integra sinergicamente il Deep Reinforcement Learning (DRL) con il Model Predictive Control (MPC) per migliorare l'autonomia e l'efficienza. Il contributo principale è un agente DRL addestrato per regolare dinamicamente in tempo reale i parametri chiave dell'MPC, in particolare l'orizzonte di predizione, il passo di controllo e le matrici di peso per lo stato e il controllo. Questo approccio sfrutta la natura predittiva e la capacità di gestione dei vincoli dell'MPC, mitigandone al contempo l'alto costo computazionale e la dipendenza da una taratura manuale grazie all'apprendimento adattivo del DRL. I risultati, convalidati tramite estese simulazioni Monte Carlo ad alta fedeltà, dimostrano che l'MPC ottimizzato dal DRL migliora significativamente le prestazioni rispetto ai controllori configurati staticamente. Il sistema ibrido ottiene un'ottimizzazione sostanziale sia in termini di efficienza del propellente che di prestazioni computazionali, con un risparmio di carburante che va dal 10% al 40% e una riduzione del tempo di calcolo relativo di circa il 65%. Aspetto fondamentale, questi guadagni sono realizzati senza compromettere le garanzie di sicurezza intrinseche al framework MPC; l'agente DRL agisce come un ottimizzatore esperto, ma lo strato MPC sottostante rimane l'arbitro finale responsabile dell'applicazione di tutte le dinamiche di sistema e dei vincoli operativi. Questo lavoro fornisce la prova definitiva che questa strategia ibrida è una soluzione praticabile, robusta ed efficiente per l'implementazione in tempo reale a bordo di sistemi di guida autonoma per veicoli spaziali.

Parole chiave: Apprendimento per Rinforzo Profondo (DRL), Controllo Predittivo Basato su Modello (MPC), Operazioni di Prossimità per Veicoli Spaziali, Orbite Altamente Ellittiche (HEO), Controllo Autonomo, Sistemi di Controllo Ibridi

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

Introduction **1**

1 Theoretical Background **9**

1.1 Dynamical Model 9

1.1.1 Coordinate Reference Frames 9

1.1.2 Unperturbed Dynamical Equations 10

1.1.3 Perturbations in ECI Frame 13

1.2 Model Predictive Control (MPC) 16

1.3 Reinforcement Learning 18

1.3.1 RL Framework 18

2 Problem Formulation **25**

2.1 Case Study 25

2.1.1 Mission constraints and docking tolerances 27

2.1.2 Navigation noise 27

2.1.3 Thrust error 28

2.2 Baseline MPC formulation 29

2.2.1 MPC constraints 29

2.2.2 Cost function 31

2.3 RL training: Computational Efficiency 33

2.3.1 Action space 34

2.3.2 Observation space 34

2.3.3 Reward Shaping 35

2.3.4 Artificial Neural Network Architecture and Hyperparameters 36
2.4 RL training: Control Performance 38
2.4.1 Action Space 38
2.4.2 Observation Space 39
2.4.3 Reward Shaping 40
2.4.4 Artificial Neural Network Architecture and Hyperparameters 41

3 Numerical Results **43**
3.1 Classical MPC performance 44
3.2 RL training: Computational Efficiency 47
3.3 RL training: Control Performance 50
3.4 Benchmark via Monte Carlo Simulations 54
3.4.1 ΔV comparison 55
3.4.2 Constraint index comparison 58
3.4.3 Computational time comparison 60

4 Conclusions and future developments **65**
4.1 Conclusions 65
4.2 Future Work 68

Bibliography **71**

List of Figures **75**

List of Tables **77**

Introduction

Rendezvous and Docking overview

Rendezvous and Docking (RVD) is an important process in spaceflight, which involves the approach and connection of an active spacecraft (the chaser) with a target object. While in this study the target is assumed to be a cooperative spacecraft, the proposed framework is general and can be extended to scenarios involving uncooperative targets such as debris or tumbling satellites. RVD plays a central role for a wide range of space applications, including in-orbit servicing, crew transfers, satellite inspection, debris removal, and space station resupply. As these services become more demanding, the need for precise, autonomous, and reliable GNC (Guidance, Navigation, and Control) systems during proximity operations is increasing [1, 2].

The RVD sequence is typically divided into smaller phases: phasing, homing, closing, final approach, and mating. The present work focuses specifically on the close-range operations, generally defined as the final few hundred meters from the target, where accuracy and control authority become critical due to tight spatial constraints, safety requirements and risk of plume impingement or collision.

During the closing phase, the chaser must align with the docking corridor, often using fly-around or hold-point maneuvers to position itself along a predefined approach axis. The spacecraft must then transition into the final approach phase, typically characterized by a quasi-linear trajectory within an allowed approach cone, defined by line-of-sight (LoS) and field-of-view limitations. Navigation accuracy becomes increasingly important, often requiring vision-based sensors, LiDAR, or radar systems to ensure sufficient observability and relative pose estimation at close range [3].

Key challenges in this regime include [4, 5]:

- High-precision relative navigation under uncertainty;
- Strict adherence to safety constraints, such as LoS corridor limits, approach velocity bounds, and obstacle avoidance;

- Thrust constraints, particularly for spacecraft with low-thrust propulsion or limited attitude control;
- Plume impingement avoidance, to prevent thermal or mechanical interference with the target.

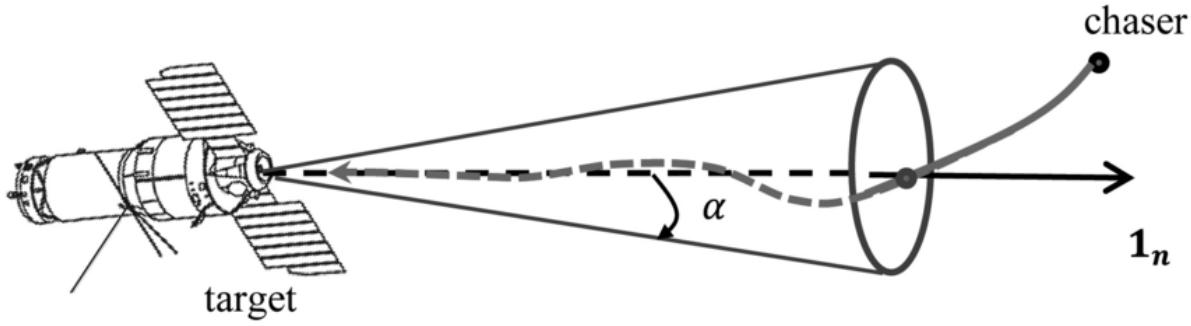

Figure 1: Schematics of a typical conic approach corridor

Historically, control during these phases relied on manual or semi-autonomous procedures. Modern approaches incorporate a wide range of closed-loop controllers to improve safety and repeatability [3, 6]. The most commonly used techniques for controlling space proximity operations include Proportional-Integral-Derivative (PID) controllers, which are typically employed for simple tracking tasks [7]; Linear Quadratic Regulators (LQR), valued for their balance between performance and stability [8]; H_∞ methods, known for their robustness in the presence of model uncertainties [9]; and Sliding Mode Control (SMC), which offers strong resilience to disturbances while remaining relatively computationally efficient [10].

These control methods often face limitations when dealing with complex mission requirements and multi-constraint environments. In particular, they typically lack the ability to explicitly handle input and state constraints or to optimize performance over a future prediction horizon [11].

For increased autonomy and constraint management under disturbances, ideally, control algorithms should not only respond to sensor inputs in real time but also anticipate future states and respect mission-critical constraints. This has driven research toward more sophisticated strategies, including trajectory optimization and constraint-aware control frameworks.

In particular, Model Predictive Control (MPC) has been widely studied for its ability to optimize constrained trajectories online [11]. However, MPC presents a high computational cost, which is not ideal for spacecraft with limited on-board processing capabilities. At the same time, Reinforcement Learning (RL) has also emerged as an experience-driven

approach capable of generating adaptive control policies for complex environments [12, 13]. Nevertheless, RL methods still lack empirical robustness when it comes to strictly satisfying safety constraints and regulatory requirements, especially in safety-critical aerospace applications [14]. To address these limitations, the present work aims to combine the strengths of MPC and RL into a unified G&C framework. MPC is employed as the backbone of the controller to ensure constraint satisfaction and predictability, while RL is used to enhance performance by learning to tune key MPC parameters. This hybrid approach seeks to leverage the structure and safety guarantees of MPC with the adaptability of RL.

Highly Elliptical and Geostationary Transfer Orbits

Although much attention has been recently directed toward cislunar and planetary exploration, Earth-centric orbits continue to play a vital role in the deployment and servicing of modern space infrastructure. In particular, Highly Elliptical Orbits (HEO) are frequently used for high-resolution Earth observation, early warning systems, space weather monitoring, and as staging or transferring platforms for high-altitude missions [15]. With the increased interest in on-orbit servicing, debris removal, and inspector satellites, autonomous rendezvous and docking (ARVD) in these eccentric orbital regimes presents a growing technological need. Highly Elliptical Orbits such as the Molniya or Tundra orbits, are commonly used for regional coverage in high latitudes. Their prolonged dwell time near apogee makes them attractive for inspection or relay missions, but they also pose challenges in terms of communication delays, line-of-sight availability, and optimal maneuver timing. In these cases, rendezvous must often be executed during a narrow window near apogee, where relative motion is slowest and the chaser has more time to adjust its trajectory. Geostationary Transfer Orbits (GTO), with perigees often in LEO and apogees near geostationary altitude, are widely used for the deployment of communication satellites. Once the primary mission is complete, satellites are normally transferred to a graveyard orbit or deorbited in accordance with international regulations. If a satellite malfunctions, however, it may remain in GTO as an inactive asset or space debris, becoming a potential target for future active debris removal or servicing missions.

HEO and GTO present unique dynamical environments that make rendezvous and proximity operations more challenging compared to circular Earth orbits. These challenges arise from:

- Significant variations in orbital velocity and altitude, requiring adaptive guidance and control systems;
- Strong perturbative forces, particularly due to Earth's oblateness (J2), third-body

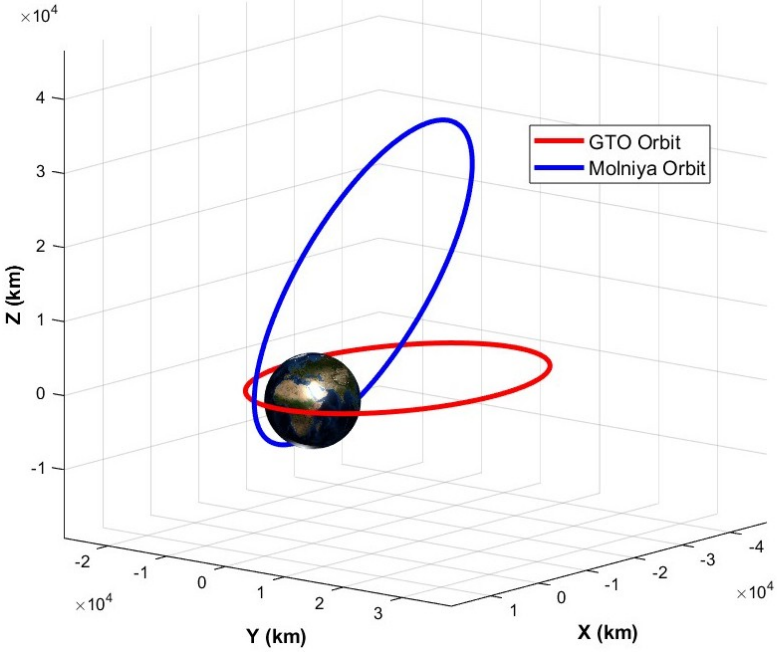

Figure 2: Geostationary Transfer Orbit (GTO) and Molniya orbit trajectories.

effects (especially the Moon in apogee-aligned orbits), and solar radiation pressure;

- Time-varying lighting and visibility conditions, which complicate the use of optical and vision-based sensors during critical phases of close-range navigation;
- High eccentricity, which leads to asymmetric dynamic behavior between perigee and apogee, requiring robust phasing and targeting strategies.

The specific constraints in these orbital regimes make the traditional assumptions of rendezvous in circular orbits—such as constant orbital parameters or relative motion linearization—difficult to apply.

Several studies have proposed tailored guidance strategies for ARVD in elliptical orbits, including methods based on Lambert targeting [16, 17, 18], relative orbital element (ROE) control [19], and impulsive rendezvous planning via multiple burns [18]. Other works [20, 21] suggest hybrid approaches that leverage both passive drift and active control, balancing performance and robustness.

Recent advances have also introduced optimal and intelligent control frameworks aimed at enhancing ARVD performance in HEO and GTO. Techniques such as Model Predictive Control (MPC), capable of explicitly handling dynamic constraints and long-horizon planning [22, 23], and Reinforcement Learning (RL) for adaptive, interaction-driven policy generation [4, 14], are emerging as promising solutions to manage the complexity and

uncertainty inherent to these orbits. These will be explored further in this thesis, with particular attention to the final approach phase under high dynamic variability.

Deep Reinforcement Learning for MPC Optimization

In recent years, Deep Reinforcement Learning (DRL) has emerged as a powerful tool across various control domains, including robotics, automotive systems, and aerospace applications [12, 14, 24]. DRL combines reinforcement learning techniques with deep neural networks to address complex sequential decision-making problems in high-dimensional environments. Unlike classical control methods that rely on a detailed model and analytical derivation of control laws, DRL enables agents to learn control policies directly from data through trial-and-error interactions with the environment.

A common approach in the literature is to use DRL to generate control actions directly [4, 13]. In this setup, the trained neural policy maps system observations to actuation commands, effectively replacing traditional control architectures. While this approach allows for significant flexibility, it suffers from key limitations in safety-critical aerospace applications. In particular, deep neural networks tend to generalize poorly outside of their training distribution and lack formal guarantees on stability and constraint satisfaction [24]. As a result, end-to-end DRL controllers have not yet been considered reliable alternatives to established methods in domains that require robustness, interpretability, and predictable performance.

Due to these limitations, traditional optimal control strategies, such as Model Predictive Control, remain the preferred choice for spacecraft G&C tasks. MPC offers a structured and well-understood framework capable of enforcing state and input constraints, handling multi-input multi-output systems, and providing closed-loop optimality over a finite prediction horizon. However, one major drawback of MPC is its high computational cost, which poses challenges for real-time implementation in on-board systems with limited resources [25].

To address this issue, RL could be employed not as a replacement for MPC, but as a meta-controller that optimizes its hyperparameters, depending on mission phase, dynamic conditions, and operational constraints. This DRL-MPC approach preserves the reliability and optimality of MPC while improving its computational efficiency and adaptability [26, 27]. Several studies have explored this approach and reported promising results [27, 28, 29, 30] but, to the best of the author's knowledge, it has not yet been applied in the context of space proximity operations.

Research questions and Outline

Research Questions

This work is guided by the following key research question, supported by several sub-questions that investigate the impact of DRL-guided MPC parameter tuning. These questions are addressed through theoretical analysis and numerical experimentation, and the corresponding discussion is presented in Chapter 4.

- To what extent can Deep Reinforcement Learning be used to improve the computational efficiency and adaptability of Model Predictive Control for spacecraft proximity operations in HEO, without compromising the safety and performance guarantees traditionally associated with MPC?

  1. How does the choice of the MPC prediction horizon T_h and control timestep Δt affect the trade-off between computational load and control performance in HEO proximity operations?
  2. What is the effect of tuning the state and control weighting matrices Q and R on the stability, aggressiveness, and fuel efficiency of the resulting guidance behavior?
  3. Can DRL identify context-aware Q and R cost matrices that outperform static or heuristically selected ones, across a wide range of scenarios?
  4. How robust is the DRL-tuned MPC system to unmodeled disturbances and parameter uncertainties in various orbits? How does it compare to a fixed MPC scheme in these situations?

Outline

Theoretical Background (Chapter 1): This chapter provides a review of spacecraft proximity operations with a focus on Highly Elliptical Orbits (HEO). It presents the modeling of absolute and relative dynamics in perturbed environments and introduces the fundamentals of Model Predictive Control (MPC), Reinforcement Learning (RL), and Deep Neural Networks (DNNs).

Problem Formulation (Chapter 2): This chapter presents the study case for proximity operations and formulates the baseline MPC problem. It defines two RL training approaches: the first one emphasizes computational efficiency, while the second one emphasizes control performance. In addition, it specifies the RL agent observation and action

spaces, neural network architectures, and training hyperparameters.

Numerical Results (Chapter 3): This chapter reports the results for the baseline MPC controller and for each RL training approach described in Chapter 2. It validates control performance and computational efficiency through high-fidelity simulations and provides a comparison with benchmark scenarios using Monte Carlo analyses under varying operational conditions.

Conclusions and Future Work (Chapter 4): This chapter summarizes the findings as answers to the initial research questions. It discusses alternative methods and highlights potential extensions for future research and development.

1 Theoretical Background

This chapter lays the foundations for the development of the main problem and scenario addressed in this thesis, establishing the basis for understanding the problem formulation and interpreting the numerical results.

- **Section** 1.1: Chaser-target relative dynamics, with a focus on RVD in Highly Elliptical Orbits (HEO).
- **Section** 1.2: Classical Model Predictive Control (MPC) problem formulation, nonlinear and linear versions.
- **Section** 1.3: Fundamentals of Reinforcement Learning, focusing on Proximal Policy Optimization (PPO) algorithm and Long Short-Term Memory (LSTM) neural networks.

1.1. Dynamical Model

1.1.1. Coordinate Reference Frames

Before presenting the details of the mathematical model that simulates the dynamics of the environment, it is essential to define the reference frames in which the dynamical models will be described. The present scenario primarily involves two space objects:

- The **Chaser** (CHS): the controlled spacecraft, assumed to act as an inspection vehicle.
- The **Target** (TRG): an unknown and uncooperative object in orbit.

This thesis uses two reference frames:

- **ECI reference frame**: The Earth-Centered Inertial (ECI) frame has its origin at the Earth's center. The $\hat{I}$ axis points towards the vernal equinox, $\hat{K}$ points towards the Earth's North Pole, and $\hat{J}$ completes the right-handed triad. The inertial coordinates are defined as [X, Y, Z]

- **LVLH reference frame**: The Local Vertical Local Horizontal (LVLH) frame is a satellite-fixed frame whose origin lies at the spacecraft's center of mass (CM). In this work, a commonly used variant is adopted: $\hat{i}$ points from the satellite to the Earth's center (local vertical), $\hat{j}$ is aligned with the orbital velocity vector (local horizontal), and $\hat{k}$ is normal to the orbital plane, pointing opposite to the angular momentum vector. The LVLH coordinates are defined as [x,y,z]

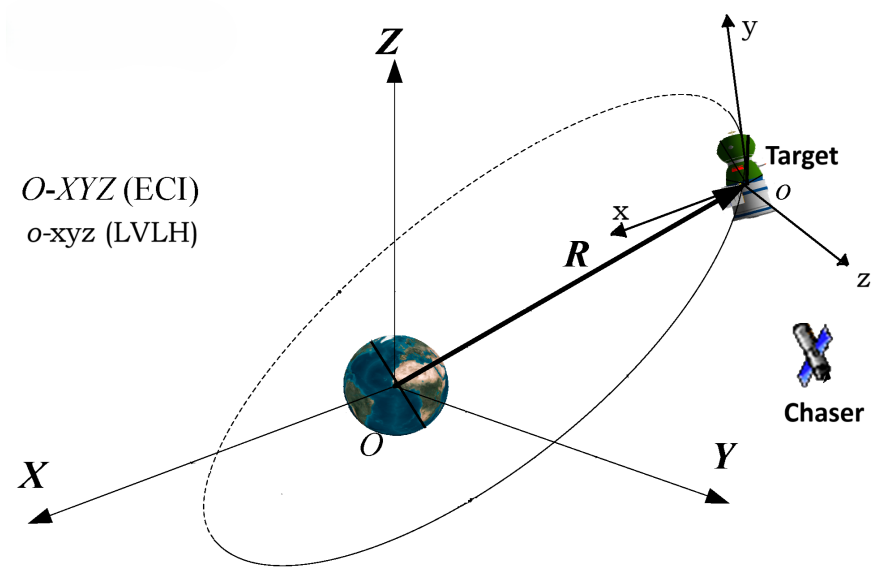

Figure 1.1: Coordinate frames in relative motion.

A fundamental assumption in this work is that the target spacecraft maintains velocity-aligned attitude, meaning that its longitudinal axis remains aligned with its velocity vector. This orientation defines the docking axis and implies that the docking interface is coaxial with the orbital path. As a result, the target's body-fixed reference frame coincides with the LVLH frame. This alignment assumption allows for further simplification of the relative dynamics formulation, as no additional rotation between the LVLH and the target body frame is required.

1.1.2. Unperturbed Dynamical Equations

Two-Body Equations of Motion

Assuming spherical bodies, no tidal forces, and that the mass of the central body is significantly larger than the orbiting spacecrafts, the following equation of motion can be used:

$$\ddot{\mathbf{r}} + \mu \frac{\mathbf{r}}{\|\mathbf{r}\|^3} = 0$$

Here, $\mathbf{r} = [r_X, r_Y, r_Z]^T$ is the position vector of the orbiting body in the ECI frame, μ is the gravitational parameter of the central body, and $\|\mathbf{r}\|$ is the Euclidean norm of the position vector.

This equation applies to both the chaser and the target spacecraft in the ECI frame:

$$\ddot{\mathbf{r}}_{\mathrm{CS}} + \mu \frac{\mathbf{r}_{\mathrm{CS}}}{\|\mathbf{r}_{\mathrm{CS}}\|^3} = 0 \tag{1.1}$$

$$\ddot{\mathbf{r}}_{\mathrm{TG}} + \mu \frac{\mathbf{r}_{\mathrm{TG}}}{\|\mathbf{r}_{\mathrm{TG}}\|^3} = 0 \tag{1.2}$$

Relative Motion and LVLH Frame

To derive the relative dynamics, the relative position vector in the inertial (ECI) frame is defined as:

$$\delta\mathbf{r} = \mathbf{r}_{\mathrm{CS}} - \mathbf{r}_{\mathrm{TG}} \tag{1.3}$$

To describe the dynamics in the LVLH frame, its axes are defined based on the target's orbital motion:

$$\hat{\mathbf{i}} = -\frac{\mathbf{r}_{\mathrm{TG}}}{\|\mathbf{r}_{\mathrm{TG}}\|}, \quad \hat{\mathbf{k}} = -\frac{\mathbf{h}_{\mathrm{TG}}}{\|\mathbf{h}_{\mathrm{TG}}\|}, \quad \hat{\mathbf{j}} = \hat{\mathbf{k}} \times \hat{\mathbf{i}}$$

Here, $\mathbf{h}_{\mathrm{TG}}$ is the angular momentum vector. The relative position in LVLH coordinates can be expressed as:

$$\boldsymbol{\rho} = x\hat{\mathbf{i}} + y\hat{\mathbf{j}} + z\hat{\mathbf{k}}$$

Subtracting Eq. (1.2) from Eq. (1.1), and substituting the relative position (Eq 1.3):

$$\delta\ddot{\mathbf{r}} = -\mu \frac{\mathbf{r}_{\mathrm{TG}} + \boldsymbol{\rho}}{\|\mathbf{r}_{\mathrm{TG}} + \boldsymbol{\rho}\|^3} + \mu \frac{\mathbf{r}_{\mathrm{TG}}}{\|\mathbf{r}_{\mathrm{TG}}\|^3} \tag{1.4}$$

To describe the relative motion in the LVLH frame, the relative inertial acceleration can be expressed using components of the non-inertial reference frame. The relationship is given by the equation:

$$\delta\ddot{\mathbf{r}} = \ddot{\boldsymbol{\rho}} + 2\boldsymbol{\omega} \times \dot{\boldsymbol{\rho}} + \dot{\boldsymbol{\omega}} \times \boldsymbol{\rho} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{\rho}) \tag{1.5}$$

Here, $\boldsymbol{\omega}$ is the angular velocity of the LVLH frame with respect to the ECI frame. The terms correspond to Coriolis, Euler, and centrifugal accelerations, respectively.

Combining Eq. (1.4) and Eq. (1.5), and assuming $\boldsymbol{\omega} = [0, 0, -\dot{\theta}_{\mathrm{TG}}]^T$, i.e., normal to the orbital plane, the following system of non-linear equations is obtained [13]:

$$\begin{cases} \ddot{x} = 2\dot{\theta}_{\mathrm{TG}}\dot{y} + \ddot{\theta}_{\mathrm{TG}}y + \dot{\theta}_{\mathrm{TG}}^2 x - \mu \dfrac{r_{\mathrm{TG}} + x}{\left[(r_{\mathrm{TG}} + x)^2 + y^2 + z^2\right]^{3/2}} + \mu \dfrac{1}{r_{\mathrm{TG}}^2} \\ \ddot{y} = -2\dot{\theta}_{\mathrm{TG}}\dot{x} - \ddot{\theta}_{\mathrm{TG}}x + \dot{\theta}_{\mathrm{TG}}^2 y - \mu \dfrac{y}{\left[(r_{\mathrm{TG}} + x)^2 + y^2 + z^2\right]^{3/2}} \\ \ddot{z} = -\mu \dfrac{z}{\left[(r_{\mathrm{TG}} + x)^2 + y^2 + z^2\right]^{3/2}} \end{cases} \tag{1.6}$$

In these equations, $\dot{\theta}_{\mathrm{TG}}$ and $\ddot{\theta}_{\mathrm{TG}}$ represent the angular velocity and angular acceleration of the target, computed from orbital parameters. The angular velocity $\dot{\theta}_{\mathrm{CHS}}$ and angular acceleration $\ddot{\theta}_{\mathrm{CHS}}$ of the chaser in its orbit can be computed from orbital mechanics principles:

$$\dot{\theta}_{\mathrm{TG}} = \frac{h_{\mathrm{TG}}}{r_{\mathrm{TG}}^2}, \qquad \ddot{\theta}_{\mathrm{TG}} = -2\frac{\mathbf{v}_{\mathrm{TG}} \cdot \mathbf{r}_{\mathrm{TG}}}{r_{\mathrm{TG}}^2}\dot{\theta}_{\mathrm{TG}}$$

While this formulation precisely captures the dynamics, solving the nonlinear model in Eq. (1.6) requires solving computationally expensive and numerically unstable equations, due to the potential for extremely large or small values arising from the fractional terms involving ρ and r_{CHS}.

Linearization of the Relative Dynamics

To address the computational challenges, a linearization approach is adopted on the assumption that the relative distance between chaser and target is significantly smaller than the distance to the central body, i.e.,

$$\frac{\rho}{r_{\mathrm{CS}}} \ll 1, \quad \frac{\rho}{r_{\mathrm{TG}}} \ll 1$$

Starting from Eq. (1.4), and applying a first-order Taylor expansion, the relative acceleration in the ECI frame is approximated as:

$$\ddot{\boldsymbol{\rho}} \approx -\frac{\mu}{r_{\mathrm{CHS}}^3}\left[\boldsymbol{\rho} - \frac{3}{r_{\mathrm{CHS}}^2}(\mathbf{r}_{\mathrm{CHS}} \cdot \boldsymbol{\rho})\mathbf{r}_{\mathrm{CHS}}\right] \tag{1.7}$$

Substituting Eq. (1.7) into Eq. (1.5), the relative motion in the LVLH frame is expressed

as a linear second-order system [31]:

$$\begin{cases} \ddot{x} = \left(\dfrac{2\mu}{r^3} + \dfrac{h^2}{r^4}\right) x - \dfrac{2(\mathbf{v}\cdot\mathbf{r})h}{r^4} y + \dfrac{2h}{r^2}\dot{y} \\ \ddot{y} = \left(\dfrac{h^2}{r^4} - \dfrac{\mu}{r^3}\right) y + \dfrac{2(\mathbf{v}\cdot\mathbf{r})h}{r^4} x - \dfrac{2h}{r^2}\dot{x} \\ \ddot{z} = -\dfrac{\mu}{r^3} z \end{cases}$$

Here, $\mathbf{r}, \mathbf{v}, r$ and h denote the position, velocity, scalar distance, and specific angular momentum of the object in the ECI frame. This system of equations can be rewritten in terms of θ from Eq. (1.7):

$$\begin{cases} \ddot{x} = \left(\dfrac{2\mu}{r^3} + \dot{\theta}^2\right) x + \ddot{\theta}\, y + 2\dot{\theta}\, \dot{y}, \\ \ddot{y} = \left(\dot{\theta}^2 - \dfrac{\mu}{r^3}\right) y - \ddot{\theta}\, x - 2\dot{\theta}\, \dot{x}, \\ \ddot{z} = -\dfrac{\mu}{r^3}\, z. \end{cases} \tag{1.8}$$

This general formulation is suitable for eccentric orbits and represents the most comprehensive unperturbed linear model [31].

1.1.3. Perturbations in ECI Frame

A realistic astrodynamics model must include several perturbative effects, each contributing differently depending on the spacecraft's altitude and orbital configuration. For this work, the considered perturbations are:

- Earth's non-spherical gravity field, up to the fifth zonal harmonic (J_5)
- Atmospheric drag
- Solar radiation pressure (SRP)

This chapter presents the mathematical formulation of the perturbations included in the ECI dynamical model.

Zonal Harmonics: J_2 to J_5

Earth's gravitational potential deviates from spherical symmetry due to oblateness and mass distribution variations. These deviations are typically captured using spherical har-

monics [31]. For equatorial symmetry, only the *zonal* harmonics (indexed by J_n) are relevant, with J_2 being the dominant term. The perturbing potential due to zonal harmonics up to J_5 is modeled as:

$$U_J(\mathbf{r}) = -\frac{\mu}{r}\sum_{n=2}^{5} J_n \left(\frac{R_e}{r}\right)^n P_n(\sin\phi)$$

where:

- R_e is the Earth's mean equatorial radius (6378.137 km)
- ϕ is the geocentric latitude
- $P_n(\sin\phi)$ are the Legendre polynomials:

$$\begin{aligned}
P_2(\sin\phi) &= \frac{1}{2}(3\sin^2\phi - 1)\\
P_3(\sin\phi) &= \frac{1}{2}(5\sin^3\phi - 3\sin\phi)\\
P_4(\sin\phi) &= \frac{1}{8}(35\sin^4\phi - 30\sin^2\phi + 3)\\
P_5(\sin\phi) &= \frac{1}{8}(63\sin^5\phi - 70\sin^3\phi + 15\sin\phi)
\end{aligned}$$

The corresponding perturbing acceleration is obtained by computing the negative gradient of the potential:

$$\mathbf{a}_J = -\nabla U_J(\mathbf{r})$$

In this work, the gradient is evaluated numerically using a central difference approximation:

$$[\mathbf{a}_J]_i = -\frac{U_J(\mathbf{r}+\varepsilon\mathbf{e}_i) - U_J(\mathbf{r}-\varepsilon\mathbf{e}_i)}{2\varepsilon}$$

where $\mathbf{e}_i$ is the i-th Cartesian unit vector and ε is a small step size (ε=0.1 km).

Atmospheric Drag

At LEO altitudes, corresponding to the perigee of some HEO, the residual atmosphere results in a drag acceleration, given by [32]:

$$\mathbf{a}_{\text{drag}} = -\frac{1}{2}C_D\frac{A}{m}\rho(h)\|\mathbf{v}\|\mathbf{v}$$

where:

- C_D is the drag coefficient (assumed 2.2, representative of a worst-case blunt-body configuration)
- A/m is the area-to-mass ratio, computed assuming a cubic satellite of side length $l = 1$ m and mass 300 kg
- $\mathbf{v}$ is the velocity of the spacecraft in ECI frame
- $\rho(h)$ is the atmospheric density at altitude $h = \|\mathbf{r}\| - R_e$

The density $\rho(h)$ is interpolated from tabulated data [33] using a logarithmic fit over altitudes up to 1000 km. Beyond this limit, drag is neglected.

Solar Radiation Pressure (SRP)

Photons from the Sun exert pressure on exposed spacecraft surfaces. This effect is modeled as a function of the distance to the Sun and the orientation, reflectivity, and absorptivity of the spacecraft surfaces relative to the incoming solar radiation [32]:

$$\mathbf{a}_{\text{SRP}} = -\frac{S}{c}\left(\frac{\text{AU}}{r_{\text{sun}}}\right)^2 C_r \frac{A}{m}\hat{\mathbf{r}}_{\text{sun}}$$

where:

- S is the solar constant (1361 W/m^2)
- c is the speed of light in vacuum
- AU is the astronomical unit
- r_{sun} is the distance between the spacecraft and the Sun
- $\hat{\mathbf{r}}_{\text{sun}}$ is the unit vector from the Sun to the spacecraft (generated randomly, on the ecliptic plane)
- C_r is the reflectivity coefficient (assumed 1.5)
- A/m is the spacecraft's area-to-mass ratio

In a more refined model, the SRP acceleration would depend on the cosine of the angle between the incident sunlight direction and the spacecraft surface normal. However, in this work, such angular dependence is neglected by assuming a worst-case scenario where the Sun's rays strike the surface perpendicularly.

Total Perturbing Acceleration

The total perturbing acceleration applied to the spacecraft in the ECI frame is the sum of all individual contributions:

$$\mathbf{a}_{\text{pert}} = \mathbf{a}_J + \mathbf{a}_{\text{drag}} + \mathbf{a}_{\text{SRP}}$$

These perturbative forces are recomputed at every integration step and are used for the propagation of both the target and chaser spacecrafts.

1.2. Model Predictive Control (MPC)

Model Predictive Control (MPC) is an advanced control strategy that uses an explicit dynamical model of the system to predict future behavior and optimize control actions accordingly. Unlike traditional feedback controllers such as PID or LQR, MPC solves an optimization problem online at each time step, taking into account system constraints and a defined performance choice [34, 25, 35].

A key distinction between MPC and classical optimal guidance algorithms is the feedback nature of MPC. While traditional optimal guidance strategies compute an open-loop trajectory based on initial conditions and execute it without updates, MPC is a closed-loop control method, solving the optimization problem at each step using the most recent state measurement. This closed-loop optimality enables MPC to handle disturbances and model mismatch robustly, but is also the reason for the high computational demand.

The core idea of MPC is to find the optimal control sequence over a finite prediction horizon, apply only the first control input, and repeat the process at the next time step.

Nonlinear MPC Formulation

At each time step t, MPC solves the following optimization problem [34]:

$$\begin{aligned}
\min_{\mathbf{u}_0,\ldots,\mathbf{u}_{N-1}} \quad & \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N) \\
\text{subject to} \quad & \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \\
& \mathbf{x}_k \in \mathcal{X}, \quad \mathbf{u}_k \in \mathcal{U}, \\
& \mathbf{x}_0 = \mathbf{x}(t),
\end{aligned}$$

Here:

- $\mathbf{x}_k \in \mathbb{R}^n$ is the system state at prediction step k
- $\mathbf{u}_k \in \mathbb{R}^m$ is the control input
- $f(\cdot)$ is the (possibly nonlinear) system dynamics
- $\ell(\cdot), \ell_f(\cdot)$ are the stage and terminal cost functions
- $\mathcal{X}$,$\mathcal{U}$ define the admissible sets for state and input
- N is the prediction horizon.

Linear MPC Formulation

In many practical applications, the system dynamics can be reasonably approximated as linear time-invariant (LTI), leading to a much less computationally taxing Quadratic Programming (QP) problem. In this case, the dynamics can be written as:

$$\mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k,$$

where $A_d \in \mathbb{R}^{n \times n}$ and $B_d \in \mathbb{R}^{n \times m}$ are the discrete-time system matrices, obtained via numerical discretization of continuous-time dynamics [35]:

$$\begin{bmatrix} A_d & B_d \\ \mathbf{0} & \mathbf{I} \end{bmatrix} = \exp\left(\begin{bmatrix} A & B \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \Delta t \right)$$

where $\exp(\cdot)$ is the matrix exponential, and Δt is the discretization time step.

The cost function in the linear MPC case typically adopts a quadratic form:

$$J = \mathbf{x}_N^{\mathrm{T}} P \mathbf{x}_N + \sum_{k=0}^{N-1} \left(\mathbf{x}_k^{\mathrm{T}} Q \mathbf{x}_k + \mathbf{u}_k^{\mathrm{T}} R \mathbf{u}_k \right) \tag{1.9}$$

where:

- Q is the state intermediate cost matrix, which penalizes deviations in the state trajectory with respect to the target
- R is the control cost matrix, which penalizes control effort,
- P is the terminal cost matrix

The terminal cost matrix P is chosen to reduce error at the end of the prediction horizon. A common approach is to compute P as the solution to the Discrete Algebraic Riccati

Equation (DARE) derived from Linear Quadratic Regulator (LQR) theory [32]:

$$P = A_d^{\mathrm{T}} P A_d - A_d^{\mathrm{T}} P B_d (R + B_d^{\mathrm{T}} P B_d)^{-1} B_d^{\mathrm{T}} P A_d + Q.$$

The optimization problem is then expressed as:

$$\begin{aligned} \min_{\{\mathbf{x}_k, \mathbf{u}_k\}_{k=0}^{N-1}} \quad & J = \mathbf{x}_N^{\mathrm{T}} P \mathbf{x}_N + \sum_{k=0}^{N-1} \left(\mathbf{x}_k^{\mathrm{T}} Q \mathbf{x}_k + \mathbf{u}_k^{\mathrm{T}} R \mathbf{u}_k \right) \\ \text{subject to} \quad & \mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k, \\ & \mathbf{x}_k \in \mathcal{X}, \quad \mathbf{u}_k \in \mathcal{U}, \\ & \mathbf{x}_0 = \mathbf{x}(t). \end{aligned}$$

This formulation forms the basis of the MPC controller developed for the proximity operations in this work.

1.3. Reinforcement Learning

Machine learning can be categorized into three main paradigms: supervised learning, unsupervised learning, and reinforcement learning (RL). While supervised and unsupervised learning primarily deal with extracting patterns from known datasets, reinforcement learning takes a fundamentally different approach. In RL, learning is driven by interaction: an agent explores an environment, takes actions, and learns to make better decisions based on the feedback it receives [24].

Rather than learning from labeled examples, the RL agent relies on scalar reward signals that indicate the quality of its actions. The objective is to discover a strategy, called policy, that maximizes the cumulative reward over time, improving its behavior through experience.

1.3.1. RL Framework

The reinforcement learning problem is typically formalized using a Markov Decision Process (MDP) [24], defined by the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where:

- $\mathcal{S}$ is the set of all possible states,
- $\mathcal{A}$ is the set of all possible actions,
- $P(s'|s, a)$ is the transition probability to state s' given state s and action a,

- $r(s, a)$ is the reward function,
- $\gamma \in [0, 1)$ is the discount factor.

At each discrete time step t, the agent observes the current state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to a policy $\pi(a|s)$, receives a scalar reward $r_t = r(s_t, a_t)$, and transitions to a new state s_{t+1} drawn from $P(s_{t+1}|s_t, a_t)$.

The goal of the agent is to learn a policy $\pi_\theta(a|s)$, parameterized by θ, that maximizes the expected cumulative discounted reward, also known as the return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right]$$

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a widely adopted actor-critic algorithm in the family of policy gradient methods. It was first introduced to improve the stability and efficiency of policy optimization, while maintaining simplicity of implementation. This is achieved by enforcing conservative policy updates, which help mitigate the risk of collapsing performance due to overly large policy shifts, a common issue in standard policy gradient methods [36].

PPO operates within the actor-critic framework, where the agent maintains two function approximators:

- a **policy network** $\pi_\theta(a|s)$ (the *actor*) that outputs a distribution over actions given a state
- a **value function** $V^\pi(s)$ (the *critic*) that estimates the expected return from a given state.

During training, the critic is used to compute an estimate of the **advantage function** $\hat{A}_t$, which quantifies how much better (or worse) an action a_t is compared to the expected performance under the current policy:

$$\hat{A}_t = Q(s_t, a_t) - V(s_t)$$

where $Q(s_t, a_t)$ is the estimated return after taking action a_t in state s_t, and $V(s_t)$ is the estimated value of the state under the current policy. In practice, $\hat{A}_t$ can be estimated using the Generalized Advantage Estimation (GAE), which helps to reduce variance while preserving low bias.

To prevent destabilizing updates, PPO introduces a clipped surrogate objective. Instead of performing pure gradient ascent on the expected advantage-weighted log-probabilities of actions, PPO defines a surrogate loss that penalizes updates that move the new policy π_θ too far from the previous policy $\pi_{\theta_{\text{old}}}$. The surrogate loss is:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right]$$

where:

- $r_t(\theta) = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the *probability ratio* that measures how much the new policy deviates from the old policy at timestep t,
- $\hat{A}_t$ is the estimated advantage at that timestep,
- ε is a small hyperparameter that sets the threshold for allowable policy updates.

If the new policy's likelihood ratio $r_t(\theta)$ moves beyond the interval $[1-\varepsilon, 1+\varepsilon]$, the update is therefore cut off and flattened. This prevents the optimizer from exploiting large advantage estimates by overly increasing (or decreasing) the probability of specific actions, preventing potential instability.

Additionally, PPO often incorporates other loss terms such as a squared-error loss for the value function and an entropy bonus to encourage policy exploration. The full objective typically includes all three components:

$$L^{\text{PPO}} = L^{\text{CLIP}}(\theta) - c_1 \cdot \mathbb{E}_t \left[(V_\theta(s_t) - R_t)^2 \right] + c_2 \cdot \mathbb{E}_t \left[\mathcal{H}[\pi_\theta](s_t) \right]$$

where:

- R_t is the empirical return at time t,
- $\mathcal{H}[\pi_\theta](s_t)$ is the entropy of the policy, promoting exploration,
- c_1 and c_2 are weighting coefficients for the value loss and entropy bonus.

PPO is praised for its empirical robustness, simplicity of implementation, and ease of tuning compared to more complex methods like Trust Region Policy Optimization (TRPO) [36]. It has become a standard baseline in reinforcement learning benchmarks and is particularly effective in continuous control problems, making it a good candidate for the scope of this work.

Recurrent Neural Networks and LSTM

Many real-world control problems are inherently partially observable, meaning that the agent does not have access to the complete state of the environment at each time step. Instead, it must make decisions based on a sequence of noisy or incomplete observations, which may only provide indirect information about the underlying dynamics. In such scenarios, traditional feedforward networks, which process each observation independently, often fall short.

Recurrent Neural Networks (RNNs), on the other hand, are explicitly designed to handle temporal sequences and are well-suited for environments where the temporal evolution of observations carries meaningful information [37]. By maintaining a hidden state, updated at each time step, RNNs can retain a form of memory that captures the history of the observed data. This allows the model to infer unobserved variables and to recognize patterns that unfold over time.

In the context of RL for control tasks, RNNs enable the policy or value function to incorporate past information into decision-making, which can significantly improve performance in partially observable or dynamic environments [24].

A widely used variant of RNNs in reinforcement learning is the Long Short-Term Memory (LSTM) network [38]. LSTMs were specifically designed to address some of the limitations of standard RNNs, most notably the issues of vanishing and exploding gradients [37], which hinder the learning of long-term dependencies in sequential data.

An LSTM unit maintains two internal states at each time step: the *hidden state* h_t, which is used for output and passed forward in time, and the *cell state* c_t, which acts as a long-term memory. These two states work together to enable the network to selectively remember or forget information over long sequences.

The update of the LSTM unit is governed by the following equations:

$$
\begin{aligned}
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
\tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

In these equations:

- x_t is the input vector at time step t
- h_t is the hidden state (or short-term memory)
- c_t is the cell state (or long-term memory) that carries accumulated information through time
- f_t, i_t, and o_t are the forget, input, and output gates, respectively
- $\tilde{c}_t$ is the candidate value to add to the cell state
- W, U are learnable weight matrices that linearly transform the input x_t and the previous hidden state h_{t-1}
- b are learnable bias vectors
- σ denotes the sigmoid activation function
- tanh is the hyperbolic tangent function
- $\odot$ denotes element-wise multiplication.

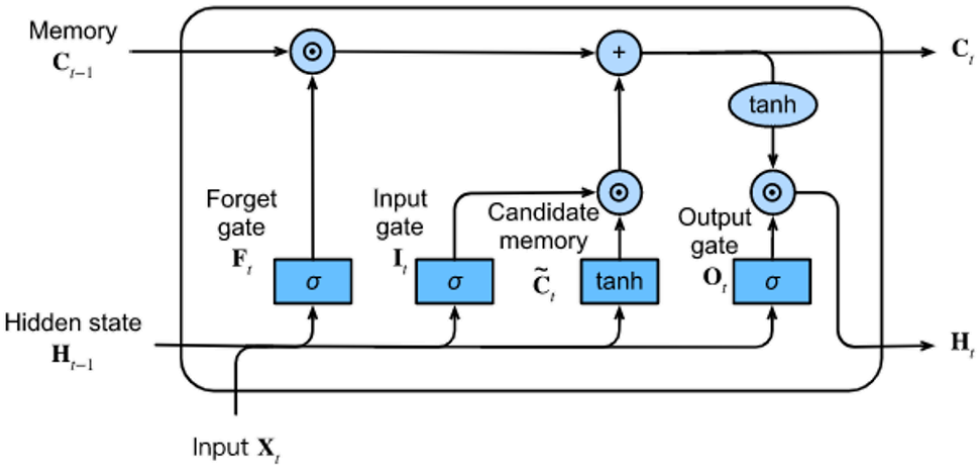

Figure 1.2: Diagram of an LSTM unit showing the flow of data through the cell and gates.

The gates control the flow of information. The forget gate f_t determines which parts of the previous cell state c_{t-1} should be discarded. The input gate i_t regulates how much of the candidate cell state $\tilde{c}_t$ should be added, and the cell state c_t is updated accordingly. The output gate o_t controls how much of the updated cell state should influence the hidden state h_t, which in turn is used for both the output and the recurrent connections.

In reinforcement learning, the hidden state h_t often serves as the input to a fully connected layer (or multiple layers) that ultimately outputs either:

- a probability distribution over actions (in policy-based methods like PPO),
- a value estimate (in value-based or actor-critic methods),
- both, in the case of actor-critic architectures.

Thus, LSTMs enable the policy or value function to be affected not just by the current observation x_t, but also by the accumulated history embedded in the hidden and cell states.

2 Problem Formulation

This chapter presents the problem formulation for the development of a Model Predictive Control (MPC) framework from first principles and its integration with Reinforcement Learning (RL) in the context of autonomous spacecraft final approach and docking.

The content is organized into four main parts:

- **Case Study** (Section 2.1):
  Definition of the docking scenario, operating conditions, and relevant constraints.
- **MPC Formulation** (Section 2.2):
  Description of MPC architecture, including cost functions and constraint handling.
- **RL training: Computational Efficiency** (Section 2.3):
  Application of RL to increase on-board computational efficiency while preserving MPC performance.
- **RL training: Control Performance** (Section 2.4):
  Use of RL to reduce fuel consumption and improve constraint satisfaction via parameter optimization.

2.1. Case Study

The scenario used for designing the MPC controller and training the RL agent is set in a GTO orbit, with orbital parameters detailed in Table 2.1. The true anomaly θ is uniformly sampled between 0° and 360°, as the MPC algorithm must be flexible enough to operate in all points of the HEO.

Parameter	Symbol	Value	Unit
Semi-major axis	a	24000	km
Eccentricity	e	0.73	-
Inclination	i	0	degrees
RAAN	Ω	0	degrees
Argument of perigee	ω	0	degrees

Table 2.1: Orbital parameters of the GTO used for training.

For each simulated episode, the chaser spacecraft starts at a hold point located outside a hypothetical hold-out sphere of radius R=100m. The initial relative distance from the target is randomly sampled between 100 and 300 m, with a maximum angular deviation of 9° from the docking axis. The initial relative velocity is randomly sampled from a Gaussian distribution with a mean of zero and a standard deviation of $0.1m/s$ along each of the three axes.
The chaser is modeled as a cubic spacecraft of medium-size. For proximity operations, the propulsion system must allow for very fine thrust control with minimal disturbances. Typically, spacecrafts of this class use cold gas or monopropellant thrusters. A relatively large maximum thrust of 5N per axis is chosen to provide sufficient control authority in contingency situations and disturbance rejection, following the design philosophy adopted in operational missions such as ESA's ATV [39] and Cygnus/Dragon proximity operations for the ISS [40]. In nominal operations the MPC controller shall demand the use of only a fraction of the available thrust, in order to optimize propellant consumption. The summary of all chaser spacecraft parameters relevant to the dynamical formulation, together with the choices of the constants of the perturbations defined in Subsection 1.1.3, are summarized in Table 2.2.

Parameter	Symbol	Value	Unit
S/C side length	l	1.0	m
S/C mass	m	300	kg
Maximum thrust	$T_{\max}$	5.0	N
Drag coefficient	c_d	2.2	-
Reflectivity coefficient	C_r	1.5	-

Table 2.2: Chaser spacecraft physical parameters.

2.1.1. Mission constraints and docking tolerances

One of the most critical constraints in RVD missions is maintaining the chaser spacecraft within the safe approach corridor. This corridor is typically quite narrow and its cross-sectional area decreases as the chaser approaches the target.

In this work, the safe zone is modeled as a truncated cone with a half angle of $\theta = 10°$, which represents a typical stringent requirement for safety and vision-based navigation in Earth-orbit missions [11, 41]. The cone axis terminates at the surface of the keep-out sphere, thus its length is set to $R = 100\,\mathrm{m}$.

The cross-section of the cone at the target-end corresponds to the dimensions prescribed by international docking standards [42]; hence, its radius is fixed at 0.1 m.

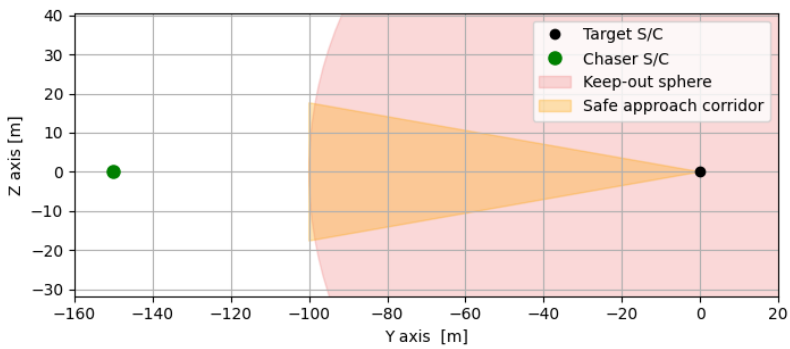

Figure 2.1: Representation of the approach corridor in the X-Y plane, LVLH frame.

Furthermore, the docking standards impose specific safety and operational constraints on the final approach conditions, summarized in Table 2.3.

Parameter	Symbol	Value	Unit
Closing axial rate	v_{axial}	0.1	m/s
Closing radial rate	v_{radial}	0.04	m/s
Closing radial misalignment	r_{misalign}	0.1	m

Table 2.3: Docking tolerances from International Docking System Standard [42]

.

2.1.2. Navigation noise

For a more realistic simulation of the controller performance, the measurements of relative position and velocity are corrupted by noise. The measurement error is modeled as

Gaussian white noise with zero mean and a standard deviation that depends on the magnitude of the relative state [43], defined as follows:

$$\sigma_{\text{pos}} = \min\left(1, \frac{e_{\text{pos}}}{40}\right)$$
$$\sigma_{\text{vel}} = \min\left(10^{-2}, \frac{e_{\text{vel}}}{30}\right)$$

where e_{pos} and e_{vel} represent the position and velocity error magnitudes, respectively.

The measurement noise vector is then given by

$$\boldsymbol{\eta} = \begin{bmatrix} \sigma_{\text{pos}}\, \mathbf{w}_{\text{pos}} \\ \sigma_{\text{vel}}\, \mathbf{w}_{\text{vel}} \end{bmatrix}$$

where $\mathbf{w}_{\text{pos}}$ and $\mathbf{w}_{\text{vel}}$ are independent Gaussian white noise vectors of dimension 3 with zero mean and unit variance.

This model ensures that the noise intensity scales with the error magnitude, to emulate the fact that some sensors (like vision-based or LIDAR) may exhibit noise that depends on distance or signal quality [43], but is capped to avoid unreasonably large disturbances in the measurements.

2.1.3. Thrust error

Errors in thrust pointing and magnitude, although not classified as external perturbations, can significantly influence the real spacecraft dynamics [32, 44]. In this study, such errors are simulated by perturbing the nominal control thrust vector as follows.

For each component u_i of the nominal thrust vector $\mathbf{u}$, the perturbed component $\tilde{\mathbf{u}}_i$ is computed as:

$$u_i' = \alpha_i\, u_i, \quad \alpha_i \sim \mathcal{U}(0.97, 1.03),$$
$$\tilde{\mathbf{u}}_i = \mathbf{R}(\theta_i)\,(u_i' \mathbf{e}_i), \quad \theta_i \sim \mathcal{N}(0, \sigma_\theta^2),$$

where α_i is the magnitude scaling factor drawn from a uniform distribution, θ_i is the pointing error angle sampled from a zero-mean Gaussian with standard deviation $\sigma_\theta = 3$, $\mathbf{e}_i$ is the unit vector along the i-th axis, and $\mathbf{R}(\theta_i)$ is a rotation matrix.

2.2. Baseline MPC formulation

The core architecture of the MPC controller developed for this work follows the standard linear MPC structure described in Subsection 1.2, but with some modifications for the specific problem at hand. The step size for the classical MPC is $dt = 1.5s$, slightly larger than non-optimal controllers like PID in similar applications [11, 45], and the prediction horizon is $T_h = 15s$, to strike a balance between long-term prediction and small computational load. Therefore, the number of steps in each horizon is $N = \frac{T_h}{dt} = 10$.

2.2.1. MPC constraints

Position constraints

Position constraints are embedded into the MPC formulation to ensure that the spacecraft remains within the designated conical region. The exact mathematical formulation of the region, however, results in nonlinear inequalities, which are unsuitable for a linear MPC.

To obtain linear constraints, the conical region is replaced by a truncated pyramid. The same semi-aperture of the pyramid must be lower than that of the cone to be fully inside the safety region, therefore it must be set to $\theta = 7.15°$. The position constraints for the pyramid are:

$$\begin{cases} z_k \leq \tan\alpha\ y_k + \varepsilon_{z,k}, \\ z_k \geq -\tan\alpha\ y_k - \varepsilon_{z,k}, \\ x_k \leq \tan\alpha\ y_k + \varepsilon_{y,k}, \\ x_k \geq -\tan\alpha\ y_k - \varepsilon_{y,k}, \end{cases} \qquad \forall k \in \{0, 1, \ldots, N-1\}.$$

Here, (x_k, y_k, z_k) are coordinates in the LVLH frame at step k within the MPC prediciton. $\varepsilon_{z,k}$ and $\varepsilon_{y,k}$ are slack variables whose contribution to the cost is weighted as:

$$J_{slack} \ = \ k_s\big(\varepsilon_{z,k} + \varepsilon_{y,k}\big).$$

with $k_s = 10^6$. Slack variables $\varepsilon_{z,k}$ and $\varepsilon_{y,k}$ soften the cone constraints, allowing small violations to preserve feasibility. The large weight k_s greatly penalizes these violations, encouraging them to remain minimal.

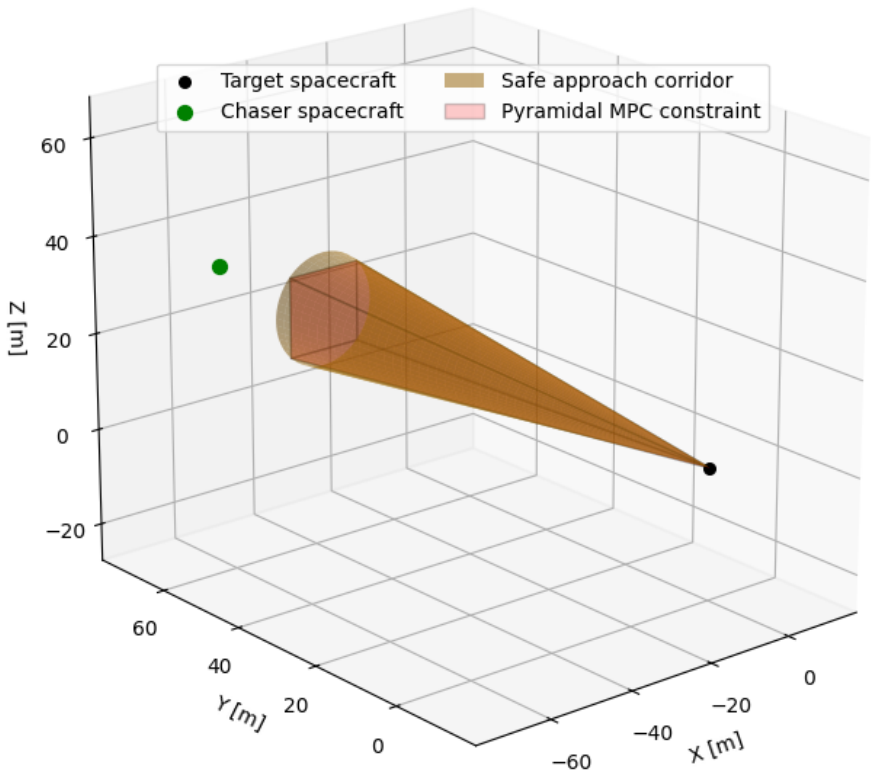

Figure 2.2: Pyramidal constraint built into the linear MPC, fully inscribed in the conical approach region. Relative LVLH frame.

Velocity constraints

Velocity constraints are primarily enforced for safety and to ensure compliance with docking standards. Let $v_{\max}$ denote the reference maximum velocity, which is defined as:

$$v_{\max} = k_v \left(1 + 200\,\text{dist_ratio}^{\,1.5}\right)$$

where `dist_ratio` is the current distance normalized by the initial one, and $k_v = 2 \times 10^{-5}$. This formulation allows higher approach speeds when the chaser is far from the target, while progressively reducing them in the proximity of the docking port to improve safety.

The constraints on the velocity components are expressed as:

$$\begin{cases} |v_y| \leq 5\, v_{\max} & \text{(longitudinal velocity)} \\ |v_z| \leq 2\, v_{\max} & \text{(perpendicular-}z\text{ component)} \\ |v_x| \leq 2\, v_{\max} & \text{(perpendicular-}x\text{ component)} \end{cases}$$

where v_y is the speed along the docking axis and v_x, v_y are the velocity components

orthogonal to it.

Constraints on the control inputs

The controller generates commands in the LVLH reference frame regardless of the spacecraft's attitude. As a result, the requested acceleration may exceed the maximum that an individual thruster can produce. This physical limitation is expressed as:

$$u_{x,k}^2 + u_{y,k}^2 + u_{z,k}^2 \leq u_{\max}^2$$

where $u_{\max}$ is the maximum acceleration achievable by a thruster. Since this inequality is non-linear and standard MPC requires linear constraints, a conservative approximation can be applied [46]. The constraints on the control actions then become:

$$-\frac{u_{\max}}{\sqrt{3}} \leq u_{x,k} \leq \frac{u_{\max}}{\sqrt{3}}, \quad -\frac{u_{\max}}{\sqrt{3}} \leq u_{y,k} \leq \frac{u_{\max}}{\sqrt{3}}, \quad -\frac{u_{\max}}{\sqrt{3}} \leq u_{z,k} \leq \frac{u_{\max}}{\sqrt{3}}$$

which are fully compatible with the linear MPC framework.

2.2.2. Cost function

The cost matrices Q and R must be tuned carefully to ensure an efficient, safe, and effective control performance. As in most MPC applications, both matrices are chosen to be diagonal for simplicity and interpretability. The state-weighting matrix Q can be conceptually decomposed into two blocks - one associated with position errors, and one with velocity errors - while the control-weighting matrix R penalizes the magnitude of control inputs, indirectly representing fuel consumption. The diagonal structure of Q and R is given explicitly by:

$$Q = \begin{bmatrix} Q_p & 0 & 0 & 0 & 0 & 0 \\ 0 & Q_p & 0 & 0 & 0 & 0 \\ 0 & 0 & Q_p & 0 & 0 & 0 \\ 0 & 0 & 0 & Q_v & 0 & 0 \\ 0 & 0 & 0 & 0 & Q_v & 0 \\ 0 & 0 & 0 & 0 & 0 & Q_v \end{bmatrix}, \quad R = \begin{bmatrix} R_c & 0 & 0 \\ 0 & R_c & 0 \\ 0 & 0 & R_c \end{bmatrix}$$

where Q_p and Q_v are the position and velocity weights, respectively, and R is the control weight.

While the velocity error and control effort typically remain within a relatively narrow

range during the maneuver, the position error can vary significantly—from hundreds of meters to a few centimeters. For this reason, Qv and R are kept constant, while the position-related weight Qp adapts dynamically.

Since the quadratic form $\mathbf{x}_k^{\mathrm{T}} Q\, \mathbf{x}_k$ implies that the contribution of position errors scales quadratically with the error magnitude, Q_p is set to increase quadratically as the chaser approaches the target:

$$Q_p = \min\left(k_p \left(\frac{1}{\text{dist_ratio}}\right)^2,\ 10^9\right)$$

where `dist_ratio` is the current distance normalized by the initial distance, and the weight is capped to avoid numerical instability. The behaviour is shown in Figure 2.3

The values of Q_v, R and k_p are set to 10^5, 10^4 and 0.5 respectively, based on a trial-and-error tuning process aimed at balancing constraint satisfaction, docking success and fuel economy.

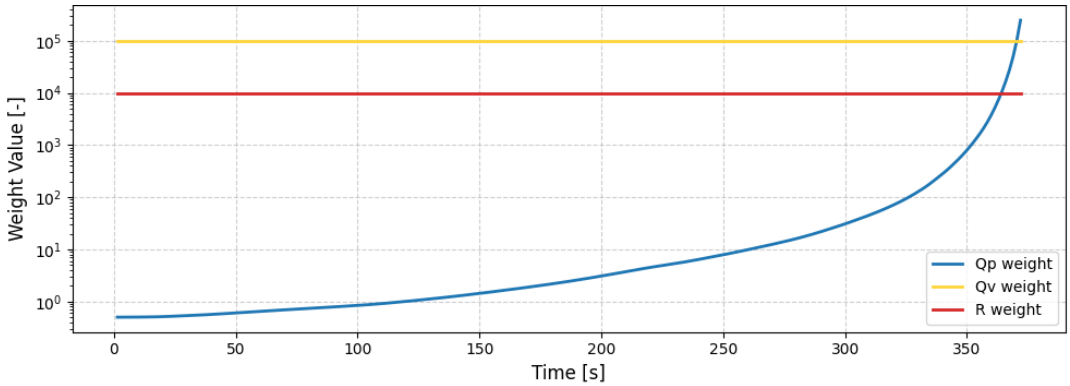

Figure 2.3: Classical MPC weights over time, showing increase of Q_p as the chaser approaches the target

To place greater emphasis on long-term convergence rather than intermediate-stage performance, the cost function in Eq. 1.9 is modified by increasing the weight of the terminal state. Specifically, the terminal cost matrix P is multiplied by N, the length of the prediction horizon. The total cost function, to be optimized at each step, is then:

$$J = \mathbf{x}_N^{\mathrm{T}} N P \mathbf{x}_N + \sum_{k=0}^{N-1} \left[\mathbf{x}_k^{\mathrm{T}} Q \mathbf{x}_k + \mathbf{u}_k^{\mathrm{T}} R \mathbf{u}_k + k_s\left(\varepsilon_{z,k} + \varepsilon_{y,k}\right)\right] \tag{2.1}$$

Here, Python's built-in function `scipy.linalg.solve_discrete_are` is used to compute the solution matrix P. For the RL training proposed in Section 2.4, however, the agent

might select poorly scaled weight matrices, which can cause the built-in solver to fail. In such cases, an approximate recursive solution is obtained by means of a Riccati iteration scheme, as outlined in Algorithm 2.1.

Algorithm 2.1 Fallback Riccati Iteration for Discrete Algebraic Riccati Equation

```
Initialize P ← Q, regularization parameter ε ← 10^{-8}
for iteration = 1 to 100 do
    Compute M ← R + B_d^⊤ P B_d
    if M is ill-conditioned then
        Regularize M ← M + εI
    end if
    Solve K ← M^{-1} B_d^⊤ P A_d
    Update P_next ← Q + A_d^⊤ (P − P B_d K) A_d
    if P_next is sufficiently close to P then
        break
    end if
    Set P ← P_next
end for
Symmetrize solution P ← ½(P + P^⊤)
```

Convergence is monitored through the difference between successive iterates, and the final solution is symmetrized to guarantee $P = P^{\mathrm{T}}$.

2.3. RL training: Computational Efficiency

The primary objective pursued through reinforcement learning is the reduction of the computational time required to solve the optimization problem within the MPC. As discussed in the introduction, a major limitation of this type of controller lies in its high computational demand, which can sometimes exceed the time available for updating the control action. This section therefore focuses on decreasing the MPC computation time, both in absolute and relative terms, in order to ensure that the controller can update the control action timely and safely.

The average computation time scales linearly with the number of steps N in the prediction horizon, where N is given by the ratio between T_h and dt. This dependence arises because N directly determines the size of the optimal control problem, both in terms of optimization variables and constraints. Very small values of N, however, lead to poor control performance and unsafe behavior [29].

To address this trade-off, the agent is tasked at each time step with selecting the values of dt and T_h that provide an effective balance between computational efficiency and overall control performance.

2.3.1. Action space

The action space is continuous (allowing for more flexibility with respect to a discrete one [47]) and two-dimensional, consisting of the scalar variables:

- dt, the control sampling time, bounded between $\mathrm{dt}_{\min} = 0.6\,\mathrm{s}$ and $\mathrm{dt}_{\max} = 4\,\mathrm{s}$,
- T_h, the prediction horizon, bounded between $T_{h,\min} = 4\,\mathrm{s}$ and $T_{h,\max} = 30\,\mathrm{s}$.

The policy outputs normalized actions $A_{dt}, A_{Th} \in [-1, 1]$, which are then linearly scaled to the corresponding physical values:

$$dt = A_{dt}\frac{\mathrm{dt}_{\max} - \mathrm{dt}_{\min}}{2} + \frac{\mathrm{dt}_{\max} + \mathrm{dt}_{\min}}{2},$$
$$T_h = A_{Th}\frac{T_{h,\max} - T_{h,\min}}{2} + \frac{T_{h,\max} + T_{h,\min}}{2}.$$

This formulation keeps the dynamics of the environment consistent and physically meaningful, while normalization of the policy output in $[-1, 1]$ helps the learning process to remain stable and converge more reliably.
In the case where T_h is not an integer multiple of dt, the value of N is defined as the greatest integer less than or equal to the ratio T_h/dt, that is,

$$N = \left\lfloor \frac{T_h}{dt} \right\rfloor$$

2.3.2. Observation space

The observation vector provided to the agent is a 9-dimensional vector designed to capture both the relative dynamics between the chaser and the target and the recent control history, which is important for learning temporal dependencies and exploiting the strengths of LSTMs. Specifically, it consists of:

- the relative state $\mathbf{x}$ between the chaser and the target, represented as a 6×1 vector including relative position and velocity,
- the action taken by the policy at the previous time step, represented as a 2×1 vector,
- the magnitude of the control input u applied at the previous time step, represented

as a scalar.

To improve learning stability and ensure consistent numerical ranges, the observations are internally normalized before being passed to the agent. The navigation noise η is added to the relative state to simulate noisy measurements. The normalized observation vector is then defined as:

$$\mathbf{obs}_k = \begin{bmatrix} \dfrac{\mathbf{x}_k + \eta}{\mathbf{x}_{\max}} \\ A_{dt}^{(k-1)} \\ A_{Th}^{(k-1)} \\ \dfrac{u_{k-1}}{u_{\max}} \end{bmatrix}$$

where $\mathbf{x}_{\max}$ represents the maximum expected magnitude of the relative state, and $u_{\max}$ is the maximum control action. Combined with the external `VecNormalize` wrapper, this normalization improves convergence and stabilizes training without introducing distortions in the physical dynamics of the environment.

2.3.3. Reward Shaping

Due to the sparse nature of the docking success signal, a dense reward shaping strategy is employed to provide continuous feedback and to guide the agent toward optimal behavior. The intermediate reward R_{int} is composed of several terms, each addressing a specific aspect of the docking task:

- **Control Effort Penalty:** Minimizing excessive use of thrusters is encouraged via

  $$R_{\text{energy}} = -w_e \frac{u}{u_{\max}} dt,$$

  where u is the control magnitude and $u_{\max}$ its maximum value. Since for each step $udt = \Delta V$, this term directly incentivizes energy-efficient maneuvers.

- **Constraint Penalty:** To ensure a safe approach along the docking corridor, deviations from the allowed approach cone are penalized. The corresponding reward term is defined as

  $$R_{\text{cone}} = -w_c \frac{r^2}{(y \cdot \tan\theta)^2} dt,$$

  Here, y represents the relative distance along the cone axis, r is the lateral displacement of the chaser from the cone axis, and θ is the cone half-angle. This term

increases as the spacecraft moves towards the edge of the approach corridor, strongly discouraging unsafe trajectories while allowing flexibility near the centerline.

- **Computation Time Penalty:** To encourage fast and efficient control updates, the reward includes a term proportional to the computation time:

$$R_{\text{time}} = -w_t \cdot t_{\text{comp}}^2,$$

where t_{comp} is the duration of the MPC control computation. Minimizing this term is the main goal of the training.

The intermediate reward is then aggregated at each time step as:

$$R_{\text{int}} = R_{\text{energy}} + R_{\text{cone}} + R_{\text{time}}.$$

The weighting coefficients were iteratevely adjusted via trial-and-error to balance the different objectives, achieving good results with $w_e = 1.25$, $w_c = 7.5$, and $w_t = 50$.

Terminal Reward The episodes end if the chaser steps outside of the safe region, if the maximum time of $15m$ is exceeded or if the docking is successful. At the end of each episode, a final reward R_{final} is given based on docking success.

$$R_{\text{final}} = \begin{cases} 100, & \text{if docking is successful} \\ -500, & \text{otherwise} \end{cases}$$

To maintain numerical stability during training, the reward is normalized using the `VecNormalize` wrapper and clipped to the range $[-10, 10]$. This prevents extreme reward values from destabilizing learning while preserving the relative importance of energy efficiency, geometric constraints, and computation time.

2.3.4. Artificial Neural Network Architecture and Hyperparameters

The neural network architecture implemented in this work is based on the `RecurrentPPO` algorithm from Stable-Baselines3, combining Long Short-Term Memory (LSTM) layers with a Multi-Layer Perceptron (MLP) feature extractor. The LSTM layers capture temporal dependencies between consecutive observations, while the MLP layers transform the sequential output of the LSTM into a fixed-size representation suitable for decision-

making.

The architecture consists of two LSTM layers with 256 hidden units each, followed by two fully connected layers with 64 neurons. Both the actor and critic networks share the same configuration. The LSTM layers employ the hyperbolic tangent (`tanh`) activation function internally, while the MLP layers also use `tanh` for their hidden activations, providing symmetry and stable gradients for bounded inputs. The use of two LSTM layers significantly improves the ability to model long-term dependencies compared to a single layer, while deeper configurations would increase training instability without measurable performance gains.

Layer	**Neurons (A/C)**	**Activation**
Input	9/9	Linear
LSTM Layer 1	256/256	Tanh
LSTM Layer 2	256/256	Tanh
MLP Layer 1	64/64	Tanh
MLP Layer 2	64/64	Tanh
Output	2/1	Linear

Table 2.4: Architecture of the Actor (A) and Critic (C) networks used in the Recurrent PPO agent.

The selection of hyperparameters plays a critical role in the stability, convergence speed, and overall performance of the reinforcement learning agent. In this work, hyperparameters are tuned through empirical trial-and-error, starting from the Stable-Baselines3 defaults and progressively adapting them to the specific requirements of the docking task.

The batch size is set to 1024 to ensure that each policy update processes data from at least four complete episodes. Consequently, the number of rollout steps is fixed at $n_{\text{steps}} = 8192$, providing a sufficient number of batches for each update cycle. The number of epochs per update is $n_{\text{epochs}} = 10$. The learning rate follows a quadratic decay schedule, starting from 1×10^{-3} and decreasing as training progresses, enabling both fast early learning and fine convergence in later stages.

The discount factor is set to $\gamma = 0.995$, and the Generalized Advantage Estimation parameter to $\lambda = 0.98$, striking a balance between bias and variance in the advantage computation while accounting for delayed rewards, such as the terminal docking reward. Gradient clipping is applied with a maximum norm of 0.3 to prevent gradient explosion

in the LSTM layers. Entropy regularization, with a coefficient of 1×10^{-4}, promotes mild exploration of the action space. Finally, the target KL divergence is fixed at 0.02 to avoid overly large policy updates and to maintain training stability.

Parameter	Value	Unit
Rollout steps (n_{steps})	8192	-
Batch size	1024	-
Epochs (n_{epochs})	10	-
Learning rate schedule	$1 \times 10^{-3} \cdot (p_{\text{rem}})^2$	-
Discount factor (γ)	0.995	-
GAE lambda (λ)	0.98	-
Clip range (ϵ)	0.2	-
Entropy coefficient	1×10^{-4}	-
Target KL	0.02	-
Max grad norm	0.3	-

Table 2.5: Hyperparameters used for training the Recurrent PPO agent.

2.4. RL training: Control Performance

In this section, the objective pursued through reinforcement learning is to improve fuel efficiency while maintaining constraint satisfaction with the MPC controller. While classical tuning of the MPC weight matrices is usually performed offline and based on heuristics, the proposed approach leverages RL to dynamically adapt the cost function weights during operation.

Specifically, the agent learns to adjust the diagonal weights of the state penalty matrix Q and of the control effort penalty matrix R. This allows for an adaptive and experience-driven tuning strategy, as opposed to fixed weight selection, which often leads to suboptimal performance under varying mission conditions.

2.4.1. Action Space

The action space is continuous and two-dimensional, consisting of the scalar variables:

- Q_{pos}, position weight of the state-weighting matrix Q, bounded between $Q_{\min} = 1$ and $Q_{\max} = 10^8$,

- R_c, the diagonal control weight of matrix R, bounded between $R_{\min} = 10^6$ and $R_{\max} = 10^{12}$.

The policy outputs normalized actions $A_Q, A_R \in [-1, 1]$, which are then mapped to the corresponding MPC weights through a logarithmic scaling:

$$Q_{\text{pos}} = 10^{\left(A_Q \cdot Q_{\text{range}} + Q_{\text{avg}}\right)},$$
$$R_c = 10^{\left(A_R \cdot R_{\text{range}} + R_{\text{avg}}\right)}.$$

where Q_{range} and R_{range} define the half-width of the logarithmic domains, while Q_{avg} and R_{avg} represent their logarithmic centers:

$$Q_{\text{range}} = \tfrac{1}{2}\left(\log_{10} Q_{\max} - \log_{10} Q_{\min}\right), \qquad Q_{\text{avg}} = \tfrac{1}{2}\left(\log_{10} Q_{\max} + \log_{10} Q_{\min}\right),$$
$$R_{\text{range}} = \tfrac{1}{2}\left(\log_{10} R_{\max} - \log_{10} R_{\min}\right), \qquad R_{\text{avg}} = \tfrac{1}{2}\left(\log_{10} R_{\max} + \log_{10} R_{\min}\right).$$

This formulation ensures that the policy explores the entire range of admissible weights in log-scale, maintaining smooth learning dynamics.

The velocity weight term Q_v does not appear in the action space of the agent. This parameter was fixed to 10^3, since in the solution of an MPC optimization problem the absolute magnitude of the cost terms is not relevant - only their relative weighting matters. By excluding Q_v from the action space, the dimensionality is reduced to two. This simplification decreases the size of the ANN while retaining the essential degrees of freedom, allowing the agent to be trained more efficiently without sacrificing control performance.

2.4.2. Observation Space

The observation vector provided to the agent is the same as that described in section 2.3.2, but the previous action now consists of A_Q and A_R:

$$\mathbf{obs}_k = \begin{bmatrix} \dfrac{\mathbf{x}_k + \eta}{\mathbf{x}_{\max}} \\ A_Q^{(k-1)} \\ A_R^{(k-1)} \\ \dfrac{u_{k-1}}{u_{\max}} \end{bmatrix}$$

where $\mathbf{x}_{\max}$ represents the maximum expected magnitude of the relative state, and $u_{\max}$ is the maximum control action.

Once again, the observations are internally normalized before being passed to the agent. Combined with the external `VecNormalize` wrapper, this normalization improves convergence and stabilizes training without introducing distortions in the physical dynamics of the environment.

2.4.3. Reward Shaping

A tailored dense reward shaping strategy is adopted to guide the agent toward minimizing fuel consumption while respecting geometric and safety constraints.

The intermediate reward is defined as:

$$R_{\text{int}} = R_{\text{pos}} + R_{\text{ang}} + R_{\text{ctrl}},$$

with the following components:

- **Position Accuracy Reward:** This component is split into two contributions. The first term encourages the chaser to reduce the distance to the target along the docking axis (y):

  $$R_{\text{pos},1} = w_{\text{pos1}} \frac{(y_{\text{min}} - y)}{y_{\text{max}}}$$

  where y_{min} is the minimum docking distance achieved in the episode, y_{max} is the maximum y distance (often at $t = 0$).

  The second term rewards the spacecraft for being near the target, introducing a strong incentive when y becomes small:

  $$R_{\text{pos},2} = w_{\text{pos2}} \exp\left(-50 \frac{y}{y_{\text{max}}}\right)$$

  Together, these two terms guide the agent both in approaching the target without stopping.

- **Angular Constraint Penalty:** This term penalizes deviations from the approach cone. The variable θ denotes the angle of misalignment between the cone axis and the line of sight of the chaser spacecraft. The penalty is defined as:

  $$R_{\text{ang}} = -w_{\text{ang}}^{\text{eff}} \left(\frac{\theta}{\theta_{\text{max}}}\right)^2$$

  where the effective weight $w_{\text{ang}}^{\text{eff}}$ is adaptively scaled with the distance y. In particular, $w_{\text{ang}}^{\text{eff}}$ increases as the spacecraft approaches the target, so that angular misalignments

are tolerated at larger distances but become progressively more critical during the final docking phase.

- **Control Effort Penalty:** Excessive fuel consumption is discouraged through a penalty proportional to the normalized control magnitude:

$$R_{\text{ctrl}} = -w_{\text{ctrl}} \frac{u}{u_{\max}}$$

The coefficients $(w_{\text{pos1}}, w_{\text{pos2}}, w_{\text{ang}}, w_{\text{ctrl}})$ are tuned via trial-and-error to balance the objectives, with final values $w_{\text{pos1}} = 100$, $w_{\text{pos2}} = 1.5$, $w_{\text{ang}} = 3$, and $w_{\text{ctrl}} = 2$.

Terminal Reward The episodes end if the chaser steps outside of the safe region, if the maximum time of $15m$ is exceeded or if the docking is successful. At the end of each episode, a final reward R_{final} is given based on docking success.

$$R_{\text{final}} = \begin{cases} +100, & \text{if docking is successful,} \\ -100, & \text{otherwise} \end{cases}$$

This design favors successful and fuel-efficient docking maneuvers, while penalizing unsafe or failed attempts.

2.4.4. Artificial Neural Network Architecture and Hyperparameters

The reinforcement learning agent is implemented using the `RecurrentPPO` algorithm from Stable-Baselines3, combining LSTM layers with an MLP feature extractor. The architecture of the ANN is the same as those described in Subsection 2.3.4.

Layer	Neurons (A/C)	Activation
Input	9/9	Linear
LSTM Layer 1	256/256	Tanh
LSTM Layer 2	256/256	Tanh
MLP Layer 1	64/64	Tanh
MLP Layer 2	64/64	Tanh
Output	2/1	Linear

Table 2.6: Architecture of the Actor (A) and Critic (C) networks used in the Recurrent PPO agent.

The training hyperparameters are largely consistent with those presented in Sec. 2.3.4. n_{steps} and n_{epochs} are adjusted to account for the shorter episode lengths resulting from the choices of dt and T_h provided by the previously trained agent. Moreover, the learning rate is reduced to improve convergence in this more challenging tuning task, while the entropy coefficient is increased to encourage broader exploration of the action space.

Parameter	Value	Unit
Rollout steps (n_{steps})	9216	-
Batch size	1024	-
Epochs (n_{epochs})	15	-
Learning rate schedule	$5 \times 10^{-5} \cdot p_{\text{rem}}$	-
Discount factor (γ)	0.995	-
GAE lambda (λ)	0.98	-
Clip range (ϵ)	0.2	-
Entropy coefficient	5×10^{-4}	-
Target KL	0.02	-
Max grad norm	0.3	-

Table 2.7: Hyperparameters used for training the Recurrent PPO agent.

3 | Numerical Results

This chapter presents and discusses the numerical results obtained from the proposed control framework.

The content is organized into four main parts:

- **Classical MPC Performance** (Section 3.1):
  Evaluation of the baseline MPC controller without reinforcement learning–based parameter tuning.
- **RL Training: Computational Cost** (Section 3.2):
  Analysis of the training process with emphasis on computational requirements and convergence behavior.
- **RL Training: Control Performance** (Section 3.3):
  Assessment of the resulting controller performance and evolution of key performance indicators in a reference geostationary orbit.
- **Benchmark via Monte Carlo Simulations** (Section 3.4):
  Extensive validation of the RL-enhanced MPC across different highly elliptical orbits (HEO).

It is worth defining here the four main performance indicators used to assess the effectiveness of the controller:

- ΔV, indicating fuel efficiency, defined as

  $$\Delta V = \sum_k u_k \, dt_k,$$

  where u_k is the control input at step k and dt_k is the corresponding timestep.
- **Constraint index**, indicating how closely the spacecraft remains to the safe region boundary (lower values are better). It is defined as the weighted sum of the

normalized distance from the cone axis to the edge of the approach corridor:

$$\text{Constraint Index} = \sum_k \frac{r_k^2}{(y_k \tan\theta)^2}\, dt_k,$$

where r_k represents the radial deviations of the spacecraft from the cone axis, y_k is the longitudinal distance from the target and θ is the half-angle of the approach corridor.

- **Relative time index**, indicating the mean fraction of the timestep dt spent solving the MPC problem:

$$t_{\text{rel}} = \frac{1}{N_k} \sum_{k=1}^{N_k} \frac{\text{CPUtime}_k}{dt_k}.$$

- **Absolute time index**, defined simply as the average CPU time required to solve the MPC problem:

$$t_{\text{abs}} = \frac{1}{N_k} \sum_{k=1}^{N_k} \text{CPUtime}_k.$$

All the dynamic equations considered in this work are integrated using a custom numerical function based on the Runge-Kutta-Fehlberg 7(8) method (RKF78), which is a high-order embedded method designed to provide stable and accurate solution.

The evaluation of the performance indexes on the reference orbit presented in Sections 3.1, 3.2 and 3.3 is performed on an *AMD Ryzen 7 7730U* processor with 8 cores and a maximum clock speed of 2.0 GHz.

3.1. Classical MPC performance

In the reference orbit (orbital parameters in Table 2.1), the Classical MPC controller, described in detail in Section 2.2, successfully provides a safe and robust control law to guide the spacecraft to the target while satisfying the imposed constraints.
An example of a chaser trajectory is shown in Figure 3.1. In addition, Figure 3.2 reports the evolution of the chaser's position and velocity components in the LVLH frame, providing further insight into the transient oscillations and convergence behavior.
It can be seen that the trajectory initially evolves somewhat erratically due to the random initial velocity. As the maneuver progresses, the path is gradually straightened and the chaser approaches the target, although with some noticeable oscillations. These oscillations can be partly attributed to external perturbations and navigation noise, which lead to erroneous control actions, but also to suboptimal MPC tuning, either in terms of a

short prediction horizon, poorly scaled weighting matrices, or other factors.

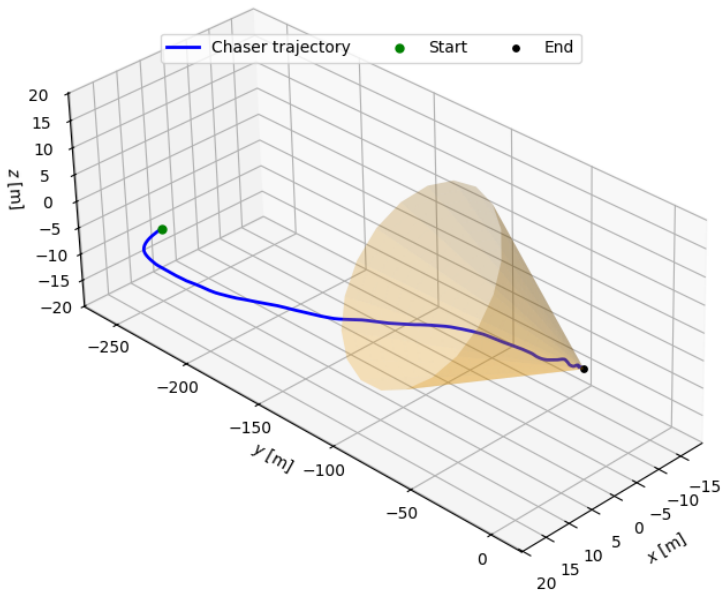

Figure 3.1: Example of a RVD trajectory with classical MPC controller, LVLH frame

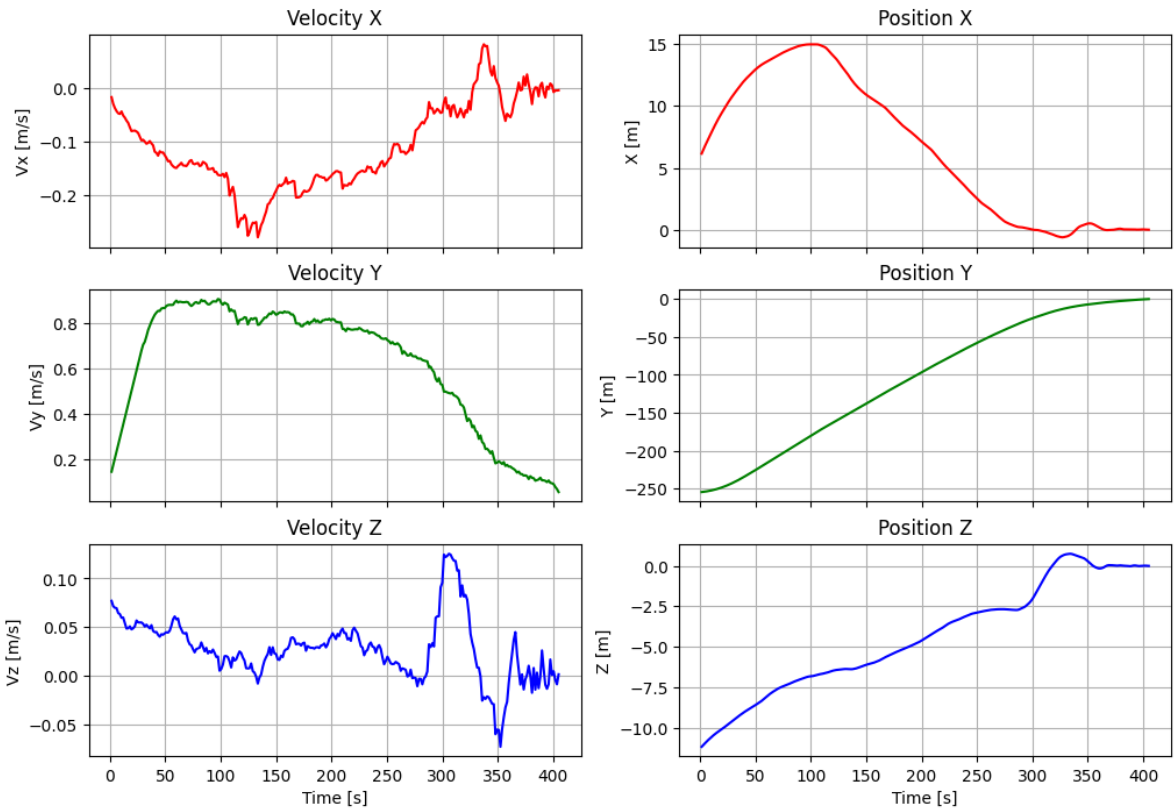

Figure 3.2: Chaser relative velocity and position, LVLH frame

The velocity components are visibly influenced by navigation noise; nevertheless, the resulting position evolution of the spacecraft remains relatively smooth. In particular, the radial components x and z converge toward zero, with the only noticeable oscillation caused by the initial velocity. In such a scenario, a simpler controller such as a PID might exhibit overcorrecting behavior, potentially inducing large and unsafe oscillations. Looking at the longitudinal component y, the velocity initially increases sharply to quickly compensate for the large position error, then gradually decreases as the spacecraft approaches the target, in order to satisfy the constraint on the maximum axial velocity at docking.

A preliminary Monte Carlo test is carried out to assess the robustness of the controller in the reference orbit. A total of 100 simulations are performed, using the random initial conditions described in Section 2.1. Figure 3.3 illustrates the rendezvous trajectories, while Figure 3.4 illustrates the performance across the main indicators, providing an overview of the controller's consistency and reliability under varying initial conditions.

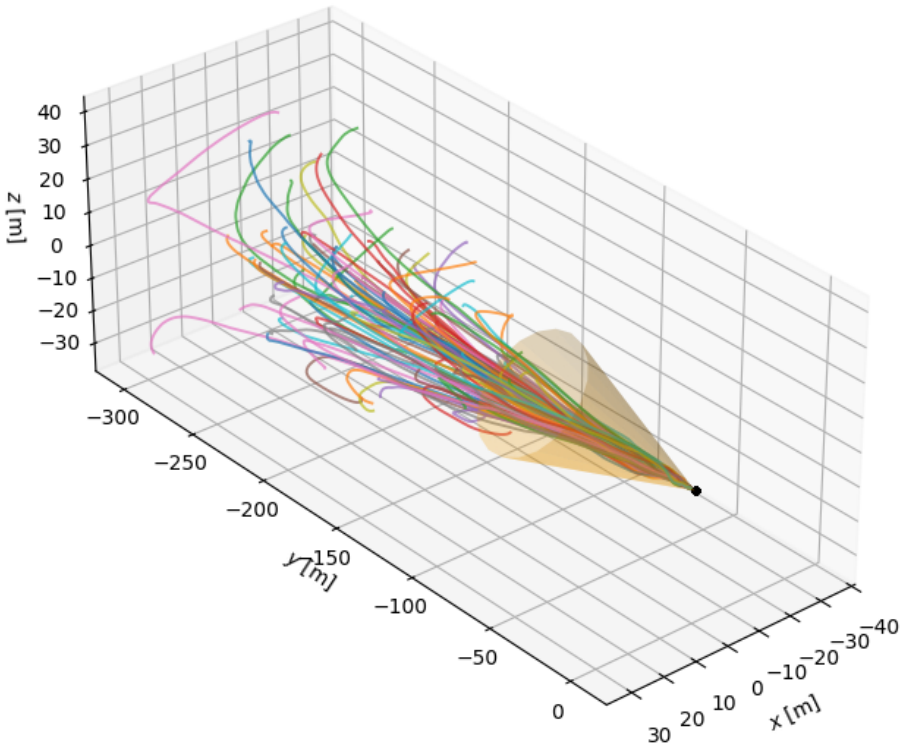

Figure 3.3: Simulated trajectories on the reference GTO orbit, classical (non-tuned) MPC, LVLH frame

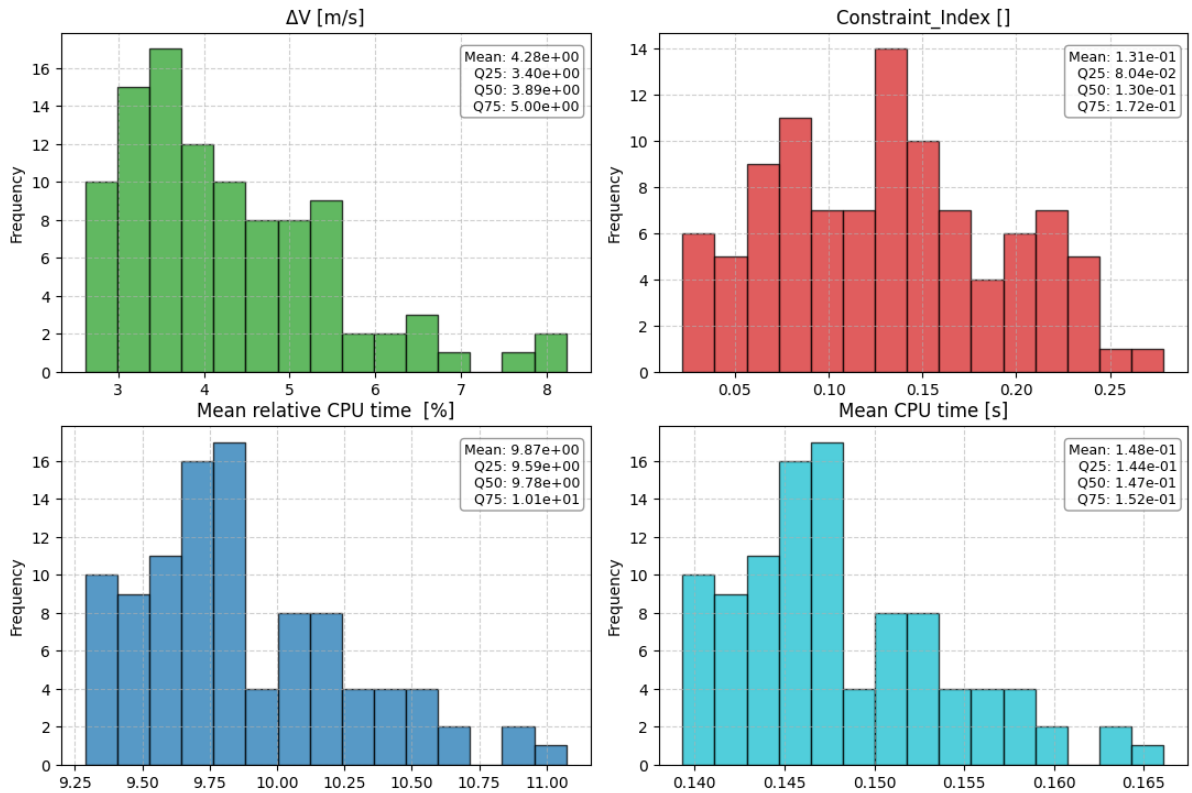

Figure 3.4: Histograms of the main performance indexes on the reference orbit, classical (non-tuned) MPC

The statistical distribution of the performance indexes reveals a larger variance in both ΔV and the constraint index, whereas the time-related indexes are more tightly clustered around their mean values. This behavior is expected: ΔV strongly depends on the initial conditions, particularly the initial distance from the target (sampled between 100 m and 300 m) and the true anomaly θ (sampled uniformly in $[0, 360°]$). Similarly, the constraint index is highly sensitive to the initial angle between the chaser and the axis of the approach cone (randomly sampled between 0° and 9°). By contrast, the time indexes are primarily dependent upon the controller parameters and are only weakly affected by the stochastic variability of the initial conditions.

3.2. RL training: Computational Efficiency

In this section, the numerical results of the RL training discussed in Section 2.3 are presented. Figure 3.5 reports the mean rollout reward obtained during each training episode.

The learning curve shows a rapid increase in reward during the initial training phase,

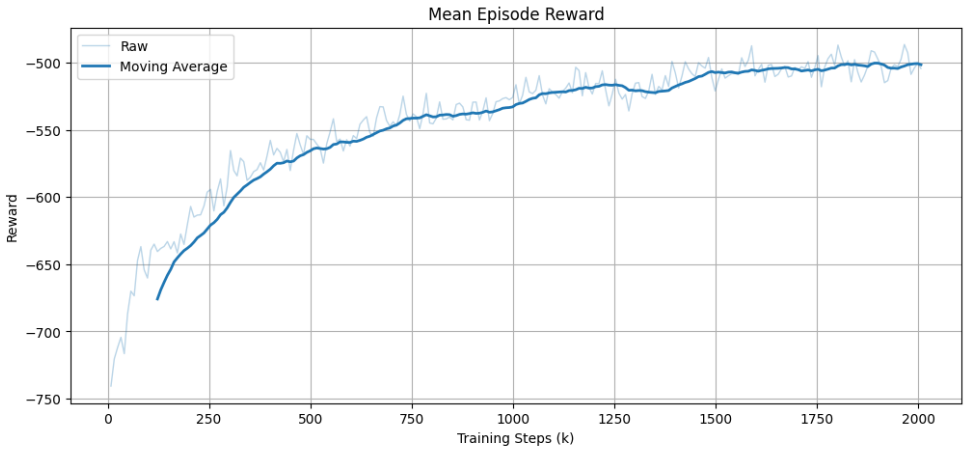

Figure 3.5: Mean rollout reward during the training phase for CPU cost

followed by a plateau that reflects the convergence of the learning process. This pattern demonstrates that the recurrent agent effectively learned to maximize the reward consistently across episodes, thereby solving the underlying MDP.

As in the case of the classical MPC, 100 Monte Carlo simulations are performed on the reference orbit to provide a preliminary performance assessment. The results for the main performance indicators are reported in Figure 3.6.

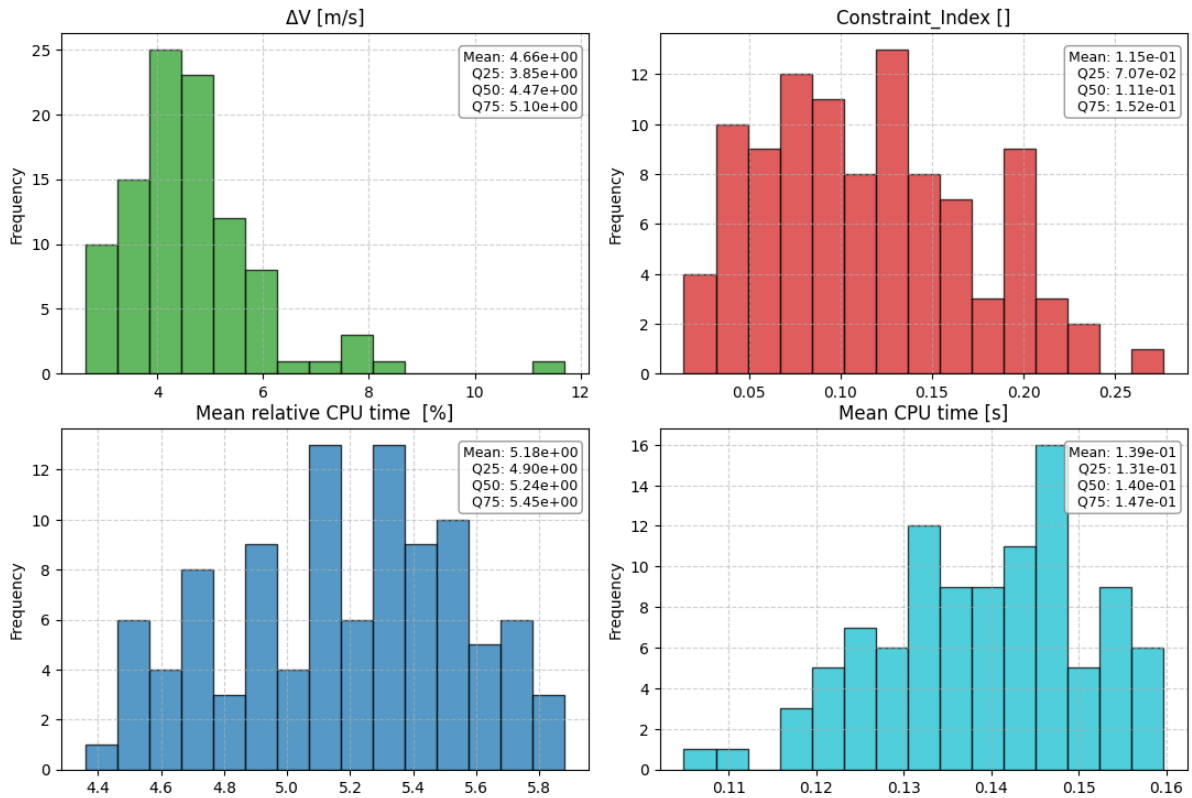

Figure 3.6: Histograms of the main performance indexes on the reference orbit, RL-tuned MPC on dt and T_h

These results clearly indicate that the primary objective of the training, and the main goal of this work, has been achieved—at least on the reference orbit. The mean computational time is substantially reduced, both in absolute terms (from $0.148s$ to $0.139s$, a reduction of 6.1%) and, most importantly, relative to the time step (from 9.87% to 5.18% of the time step).

The mean ΔV and the mean constraint index exhibit only minor variations: a slight increase of ΔV and a slight decrease of the constraint index. Importantly, neither ΔV nor the constraint index significantly deteriorated, and the maneuver was successfully completed in 100% of the simulations. Further improvements in these performance metrics are addressed in the second RL training, discussed in Section 3.3.

The same considerations made for the non-tuned MPC regarding the distribution of the indexes around their mean values also apply here, with ΔV and the constraint index exhibiting much larger variance due to their strong dependence on the random initial conditions.

The action selected by the agent to achieve these results (shown for a representative episode, as all episodes exhibit similar behavior) is reported in Figure 3.7.

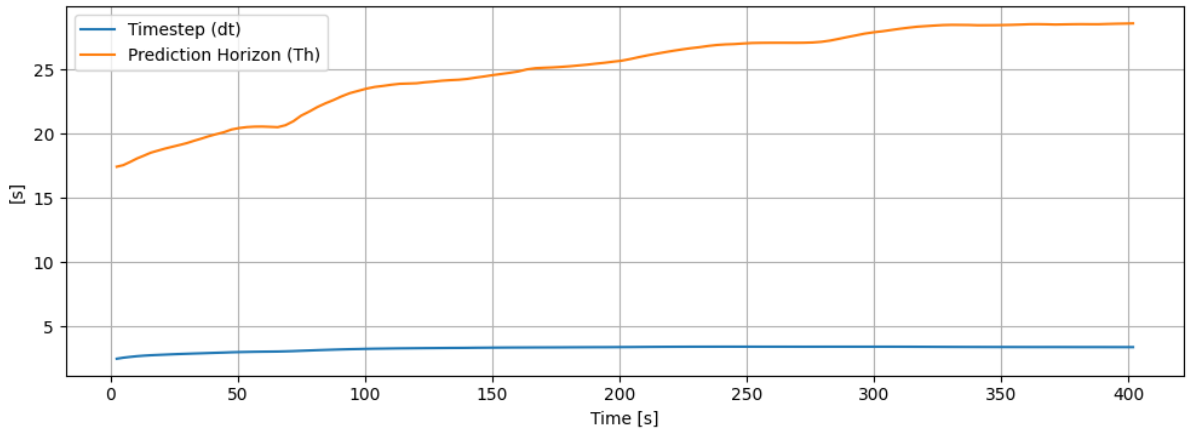

Figure 3.7: Time step and Prediction horizon provided by the agent during the RVD episode

It can be observed that both the timestep dt and the prediction horizon T_h are not fixed but vary significantly throughout the episode, with an overall increasing trend. At the beginning of the episode, the prediction horizon is relatively short, likely because the primary objective in this phase is to stabilize the velocity of the chaser, making a longer horizon unnecessary. As the distance to the target decreases and the approach corridor narrows, however, the prediction horizon must be increased. The time step dt follows a similar trend, gradually increasing together with T_h, such that N, the number of steps within each MPC optimization problem, remains limited. At the same time, dt does not increase excessively, ensuring that the update frequency remains sufficiently high to maintain safety.

3.3. RL training: Control Performance

In this section, the numerical results of the RL training discussed in Section 2.4 are presented. Figure 3.8 reports the mean rollout reward obtained during each training episode.

The learning curve is similar to that of the previous section, with a rapid increase followed by a plateau, once again reflecting the convergence of the learning process.
As in previous cases, 100 Monte Carlo simulations are performed on the reference orbit to provide a preliminary performance assessment. Figure 3.9 illustrates the rendezvous trajectories for this case.

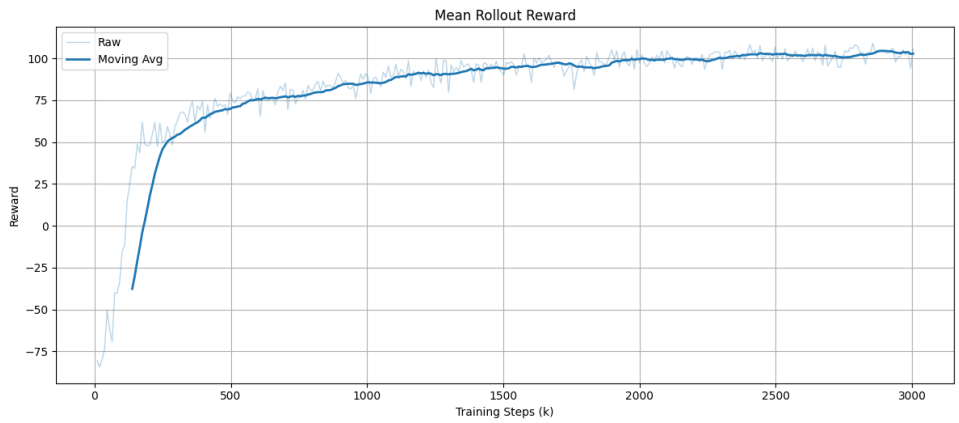

Figure 3.8: Mean rollout reward during the training for Fuel efficiency and Constraint satisfaction

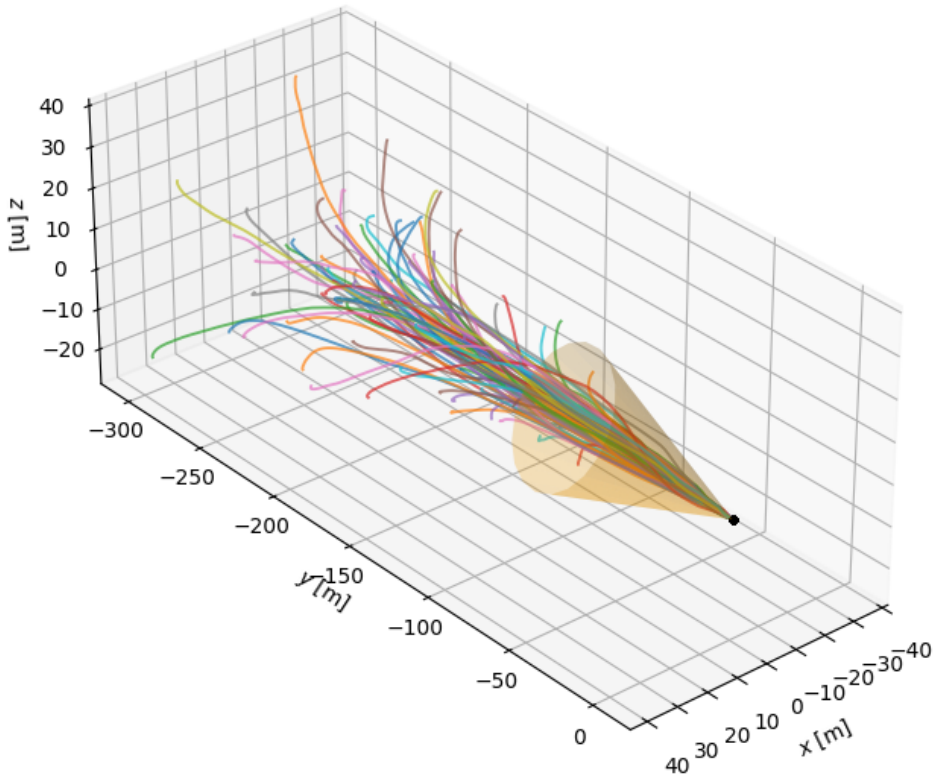

Figure 3.9: Simulated trajectories on the reference GTO orbit, RL-tuned MPC, LVLH frame

Comparing the trajectories in Figure 3.9 to those of the classical MPC controller in Figure 3.3, the most noticeable difference appears at the beginning of the simulation. Here, the controller immediately corrects the initial velocity error and steers the spacecraft toward

the docking axis. A possible explanation for this behavior is provided later in this section in the discussion of the actions of the RL agent.

The results for the main performance indicators are reported in Figure 3.10.

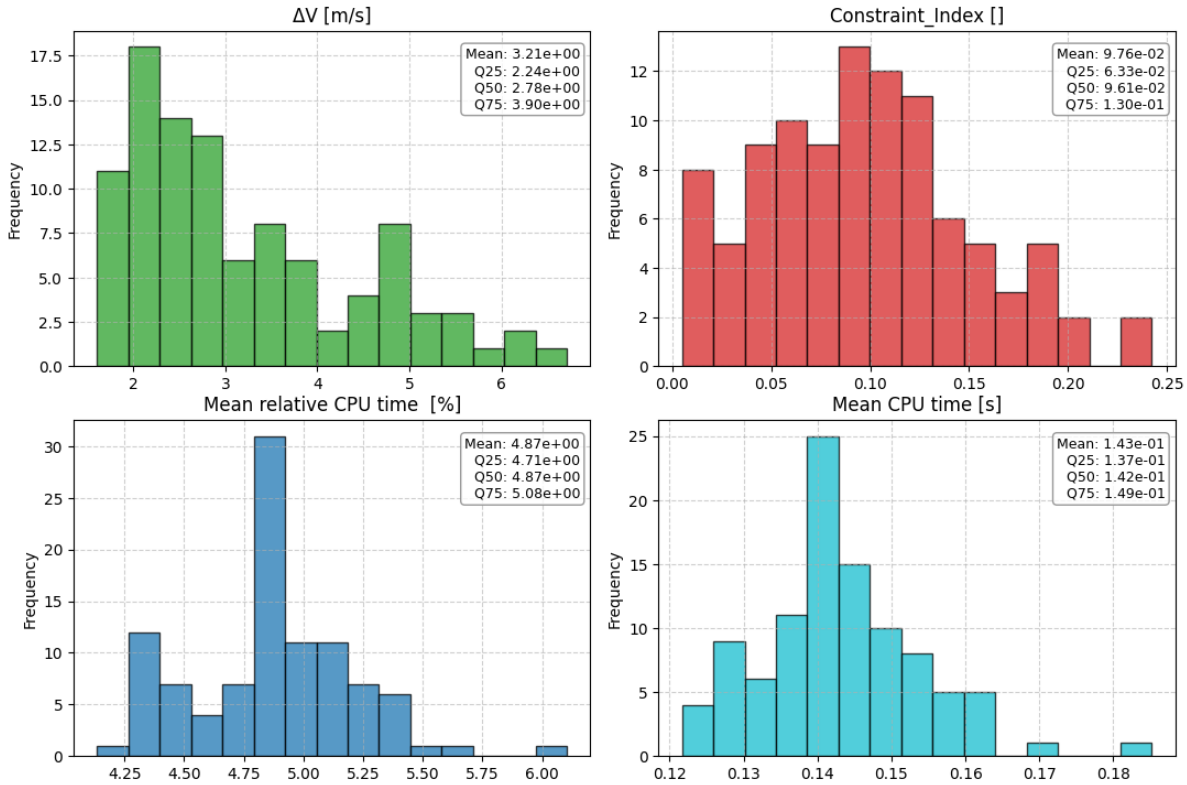

Figure 3.10: Histograms of the main performance indexes on the reference orbit, RL-tuned MPC on Q and R

These results clearly indicate that the objective of the training has been achieved, at least on the reference orbit. The mean ΔV is substantially reduced, from 4.28m/s to 3.21m/s, a reduction of 25%, proof of improved fuel efficiency.
The mean constraint index is also reduced, from 0.131 to 0.0976, indicating an improved margin of constraint satisfaction.

The computational cost indexes exhibit only minor variations with respect to the previous section, maintaining the improvements over the non-tuned controller.

The same considerations made for the non-tuned MPC regarding the distribution of the indexes around their mean values also apply here, with ΔV and the constraint index exhibiting much larger variance due to their strong dependence on the random initial conditions.

The evolution of the cost-function weights (shown for a representative episode, as all episodes exhibit similar behavior) is reported in Figure 3.11.

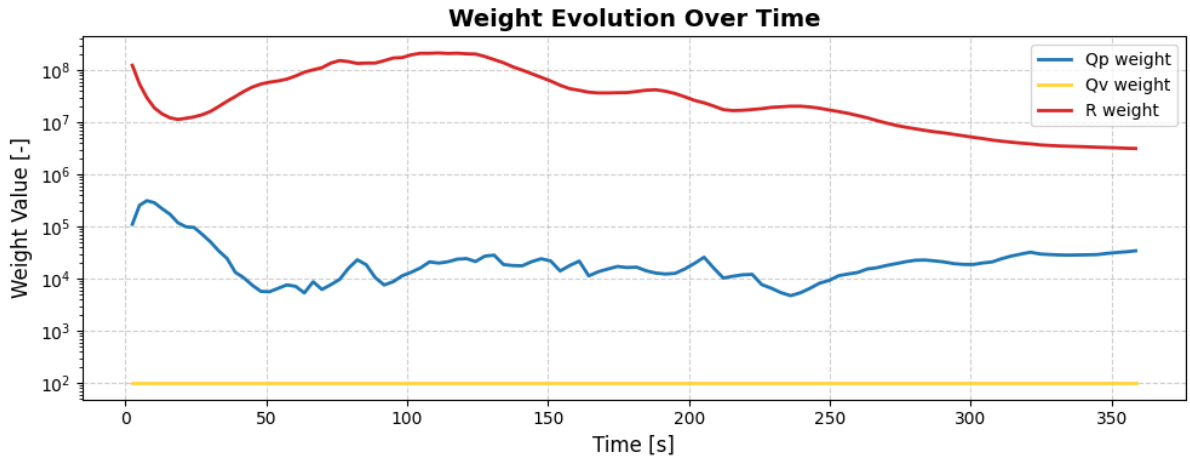

Figure 3.11: Q_p, Q_v and R values during the RVD episode, on the reference orbit, after training

As is often the case in machine learning applications for complex control tasks, it is not straightforward to identify a clear pattern behind the agent's choices. Nevertheless, some aspects of the learned behavior appear interpretable when examined along the timeline of the maneuver.

At the beginning, a sharp decrease in the R weight is observed, while Q_v remains fixed and Q_p increases. This behavior suggests that the controller prioritizes a rapid correction of the initial error, even at the expense of fuel efficiency. During the subsequent mid-phase of the RVD maneuver, R rises to its maximum value, indicating a shift towards fuel efficiency once a stable approach velocity and path have been established.

The Q_p component maintains a relatively stable value, without varying over several orders of magnitude, as in the non-tuned MPC. This trend indicates that preserving a strong incentive to reduce position error from the early stages of the maneuver proves beneficial for overall performance. Moreover, no clear correlation emerges between Q_p and R, suggesting that the agent adjusts these weights independently depending on the current phase of the maneuver.

Finally, towards the end of the approach, a gradual decrease of R is observed, implying that the controller becomes increasingly willing to trade fuel efficiency for enhanced safety during the most critical terminal phase.

3.4. Benchmark via Monte Carlo Simulations

After assessing the performance of both the tuned and non-tuned MPC controllers on the reference orbit (parameters in Table 2.1), the next step is to verify their ability to complete the same rendezvous maneuver in different orbital scenarios, and subsequently to compare their performance.

For this purpose, a set of seven representative verification orbits is considered, covering three widely used families of Earth highly elliptical orbits (HEO): three Geostationary Transfer Orbits (GTO), two Molniya orbits, and two Tundra orbits.

The exact orbital parameters are listed in Table 3.1.

Orbit	a **[km]**	e **[-]**	i **[deg]**	Ω **[deg]**	ω **[deg]**
GTO Orbits					
GTO 1 (Standard)	24414	0.7267	7.0	0	0
GTO 2 (High inclination)	24414	0.7267	28.5	0	0
GTO 3 (Low perigee)	24364	0.7303	7.0	0	0
Molniya Orbits					
Molniya 1 (Standard)	27300	0.73	63.4	0	0
Molniya 2 (Low perigee)	25448	0.73	63.4	0	0
Tundra Orbits					
Tundra 1 (Standard)	42164	0.30	63.4	0	0
Tundra 2 (High ecc.)	42164	0.40	63.4	0	0

Table 3.1: Orbital parameters of the verification orbits used for performance assessment.

The right ascension of the ascending node (Ω) and the argument of perigee (ω) are fixed to zero for all orbits. These angles are often chosen to obtain specific ground-track properties (e.g., $\omega \approx 270°$ for Molniya orbits to place apogee over the northern hemisphere). Since this work focuses on controller performance under varying eccentricities, inclinations, and semi-major axes, varying Ω and ω adds no additional insight. Fixing them to zero simplifies the analysis without affecting the generality of the results.

For each orbit, the assessment is performed at four significant true anomalies, $\theta = 0°, 90°, 180°, 270°$, corresponding to the perigee, apogee, and the two intermediate points. This selection allows evaluation of the controllers under the full range of dynamical conditions encountered along the orbit. At each point on each orbit, 100 Monte Carlo simulations are performed. The initial states of the chaser in the LVLH frame are fixed by

setting the seed of the random number generator, ensuring comparability across simulations. A total of 2,800 simulations are executed for each controller (7 orbits $\times$ 4 points per orbit $\times$ 100 simulations per point).

The simulations discussed in this section are performed on an *Intel(R) Core(TM) i7-14700K* CPU, which is a significantly faster processor than the one used for Sections 3.2 and 3.3. This explains the overall reduction in the reported time indexes. Nevertheless, the relative trends remain unchanged, and the conclusions drawn from the analysis are still valid.

3.4.1. ΔV comparison

Orbit	$\theta = 0°$	$\theta = 90°$	$\theta = 180°$	$\theta = 270°$
GTO 1	4.3451	4.0960	3.9459	4.0406
	3.8772	2.8159	2.3153	2.8506
GTO 2	4.2178	4.1035	4.0246	4.0547
	3.8652	2.8097	2.3138	2.8569
GTO 3	4.3450	4.0815	3.9598	4.0475
	3.9315	2.8173	2.3061	2.8602
Molniya 1	4.0538	4.0529	3.9627	4.0373
	3.6238	2.7448	2.3051	2.7748
Molniya 2	4.0538	4.0531	3.9924	4.0485
	3.7713	2.7781	2.3101	2.8321
Tundra 1	3.9995	4.0067	3.9559	4.0065
	2.4050	2.3570	2.3114	2.3505
Tundra 2	4.0048	3.9908	3.9589	3.9795
	2.4615	2.3639	2.3066	2.3548

Table 3.2: Mean ΔV values for non-tuned (top row) and tuned (bottom row, color-coded) MPC controllers.

Orbit	$\theta = 0°$	$\theta = 90°$	$\theta = 180°$	$\theta = 270°$
GTO 1	4.3451	4.0960	3.9459	4.0406
	3.8772	2.8159	2.3153	2.8506
GTO 2	4.2178	4.1035	4.0246	4.0547
	3.8652	2.8097	2.3138	2.8569
GTO 3	4.3450	4.0815	3.9598	4.0475
	3.9315	2.8173	2.3061	2.8602
Molniya 1	4.0538	4.0529	3.9627	4.0373
	3.6238	2.7448	2.3051	2.7748
Molniya 2	4.0538	4.0531	3.9924	4.0485
	3.7713	2.7781	2.3101	2.8321
Tundra 1	3.9995	4.0067	3.9559	4.0065
	2.4050	2.3570	2.3114	2.3505
Tundra 2	4.0048	3.9908	3.9589	3.9795
	2.4615	2.3639	2.3066	2.3548

Table 3.3: Mean ΔV values for non-tuned (top row) and tuned (bottom row, color-coded) MPC controllers.

Regarding the ΔV metric, the tuned MPC consistently outperforms the non-tuned controller, achieving higher efficiency at all orbital positions. From the values reported in Table 3.3 and the bar charts in Figures 3.12 and 3.13, it is evident that the non-tuned MPC delivers a highly uniform ΔV performance across all orbits and true anomalies. In contrast, the tuned controller exhibits fluctuations that reflect its sensitivity to the surrounding dynamical conditions.

The largest ΔV values are observed at perigee in the most eccentric orbits, where atmospheric drag amplifies perturbations and additional corrective thrust is required. Conversely, the most efficient maneuvers occur near apogee or along lower-eccentricity orbits, where perturbative forces are weaker and absolute velocities (ECI) are lower.

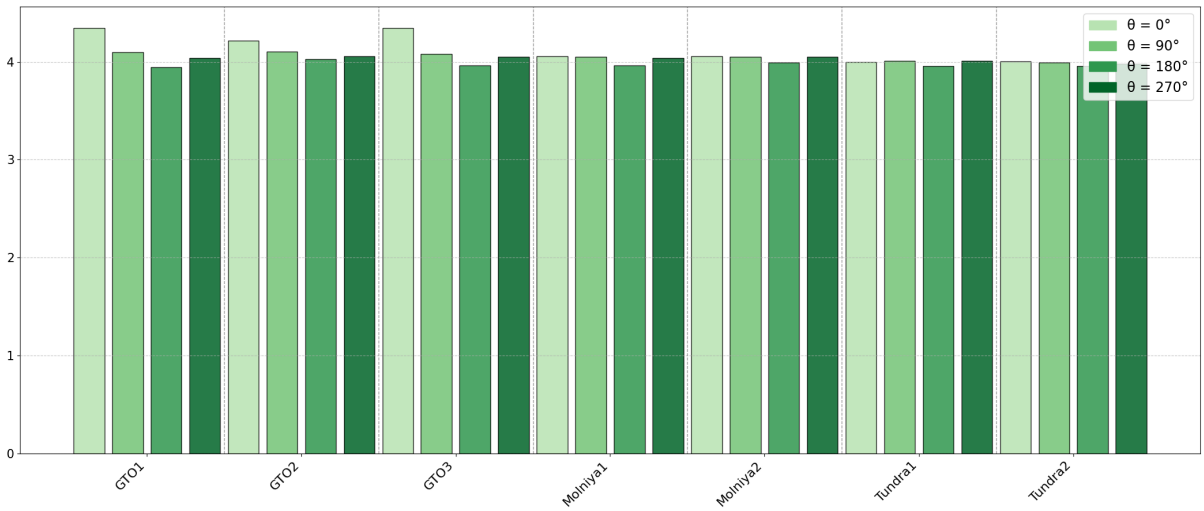

Figure 3.12: Mean values of ΔV [m/s] for the non-tuned controller, broken down by orbit and true anomaly.

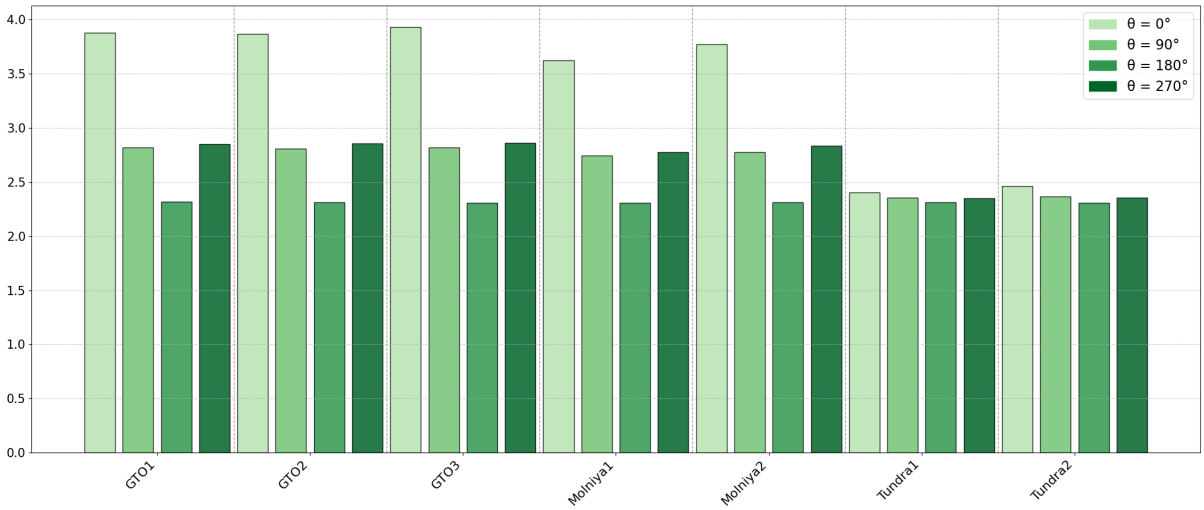

Figure 3.13: Mean values of ΔV [m/s] for the tuned controller, broken down by orbit and true anomaly.

3.4.2. Constraint index comparison

Orbit	$\theta = 0°$	$\theta = 90°$	$\theta = 180°$	$\theta = 270°$
GTO 1	0.1487	0.1194	0.1158	0.1145
	0.3874	0.1392	0.0641	0.1440
GTO 2	0.1356	0.1214	0.1133	0.1160
	0.2815	0.1337	0.0659	0.1393
GTO 3	0.1517	0.1150	0.1135	0.1177
	0.4188	0.1341	0.0649	0.1456
Molniya 1	0.1152	0.1157	0.1086	0.1129
	0.1582	0.1404	0.0636	0.1369
Molniya 2	0.1106	0.1150	0.1099	0.1156
	0.2360	0.1365	0.0656	0.1433
Tundra 1	0.1082	0.1105	0.1085	0.1100
	0.0772	0.0716	0.0637	0.0717
Tundra 2	0.1125	0.1115	0.1095	0.1088
	0.0862	0.0758	0.0642	0.0771

Table 3.4: Mean constraint index values for non-tuned (top row) and tuned (bottom row) MPC controllers. Color-coding for the tuned controller indicates performance relative to the non-tuned one: green for better, yellow for slightly worse, and orange for more than 2 times worse.

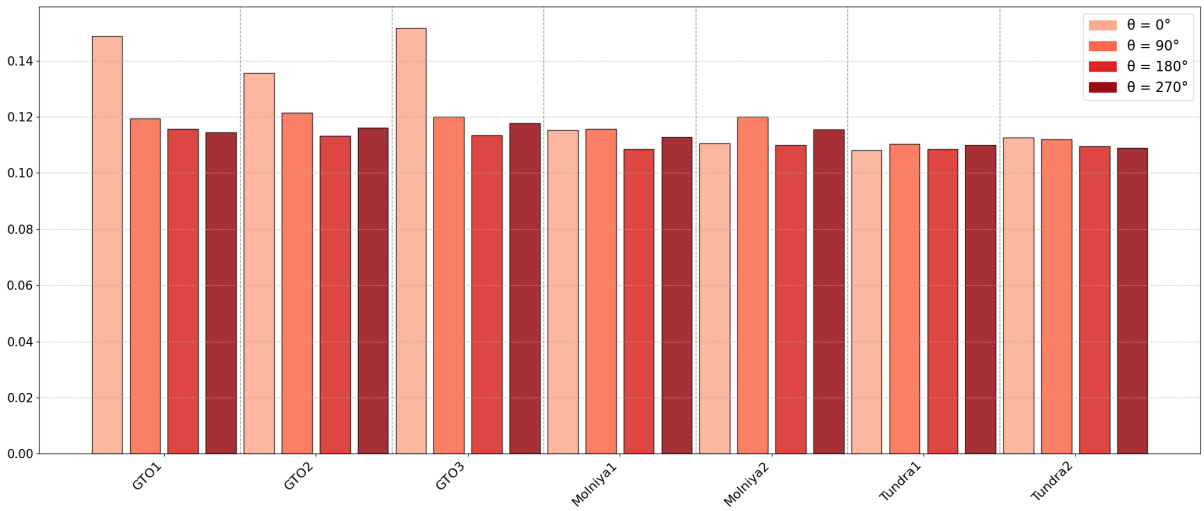

Figure 3.14: Mean values of constraint index (dimensionless) for the non-tuned controller, broken down by orbit and true anomaly.

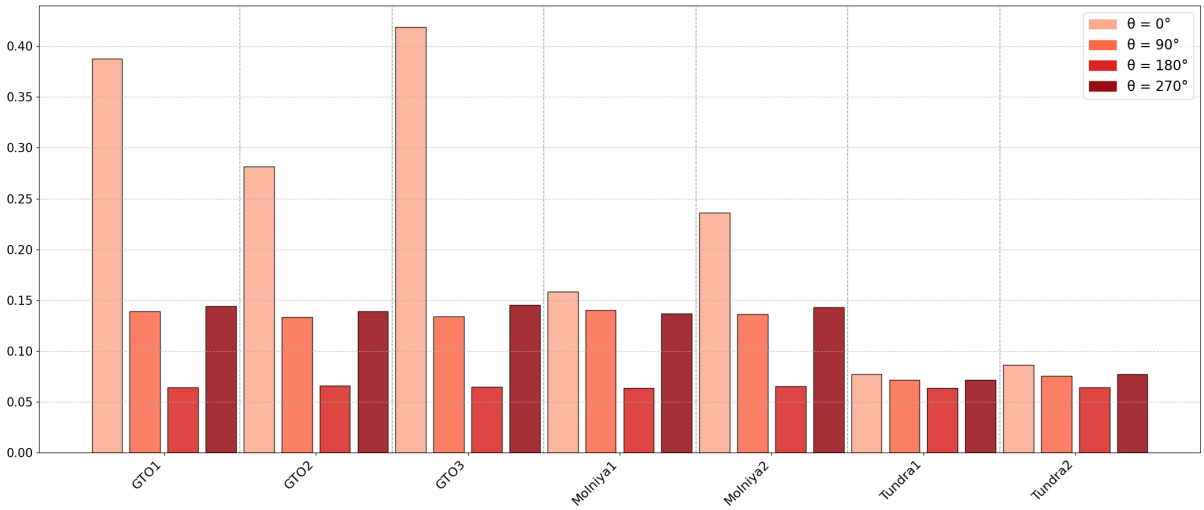

Figure 3.15: Mean values of constraint index (dimensionless) for the tuned controller, broken down by orbit and true anomaly.

Just like for the ΔV metric, the non-tuned controller maintains a remarkably consistent behavior across all orbital positions and scenarios, whereas the tuned controller exhibits a highly variable performance profile, reflecting the dynamic adjustment of its control strategy.

The tuned controller achieves a better constraint index in 13 out of the 28 test cases. The most notable degradations, highlighted in orange in Table 3.4, occur at the perigee ($\theta = 0°$) of the highest-eccentricity orbits, namely the GTOs and low-perigee Molniya orbit. This

suggests that, in the presence of strong perturbative forces such as atmospheric drag, the tuned controller prioritizes fuel efficiency (ΔV), tolerating a larger deviation of the chaser from the docking axis, which leads to a higher constraint index. A different explanation is that the relative velocity difference, resulting from the chaser being on a slightly different orbit, provides an advantage that enables a more efficient approach. The chaser spacecraft might exploit this effect by remaining slightly offset from the docking axis. Importantly, even in cases where the constraint index increases, the chaser always remains within the mission-critical approach cone, ensuring that operational safety margins are never violated.

Conversely, during apogee maneuvers ($\theta = 180°$) and in the lower-eccentricity Tundra orbits, the tuned controller outperforms the non-tuned version by $\approx 30\%$. In these more favorable orbital conditions, it is able to tighten the tracking of the docking axis without compromising fuel economy. This adaptive behavior highlights the effectiveness of the RL tuning process.

3.4.3. Computational time comparison

Orbit	$\theta = 0°$	$\theta = 90°$	$\theta = 180°$	$\theta = 270°$
GTO 1	5.2127	5.1501	5.2102	5.2280
	1.8736	1.7960	1.8158	1.7924
GTO 2	5.2441	5.2489	5.2291	5.2559
	1.8713	1.8038	1.8192	1.8089
GTO 3	5.2399	5.2663	5.2184	5.2283
	1.9040	1.7998	1.8244	1.7937
Molniya 1	5.2128	5.2469	5.2196	5.2362
	1.8315	1.7956	1.8197	1.8092
Molniya 2	5.2445	5.2525	5.2624	5.2340
	1.8287	1.7837	1.8300	1.7981
Tundra 1	5.2398	5.2157	5.2576	5.2469
	1.8104	1.8242	1.8439	1.8186
Tundra 2	5.2268	5.2469	5.2437	5.2576
	1.8021	1.8023	1.8319	1.8082

Table 3.5: Mean relative time index, t_{rel} (adimensional, percentage), for the non-tuned (top row) and tuned (bottom row) MPC controllers.

Orbit	$\theta = 0°$	$\theta = 90°$	$\theta = 180°$	$\theta = 270°$
GTO 1	0.0782	0.0780	0.0782	0.0784
	0.0617	0.0581	0.0576	0.0580
GTO 2	0.0787	0.0787	0.0784	0.0788
	0.0616	0.0584	0.0578	0.0585
GTO 3	0.0786	0.0790	0.0783	0.0784
	0.0627	0.0583	0.0579	0.0580
Molniya 1	0.0782	0.0787	0.0783	0.0785
	0.0601	0.0580	0.0577	0.0584
Molniya 2	0.0787	0.0788	0.0789	0.0785
	0.0602	0.0577	0.0581	0.0581
Tundra 1	0.0786	0.0782	0.0789	0.0787
	0.0577	0.0581	0.0585	0.0579
Tundra 2	0.0784	0.0787	0.0787	0.0789
	0.0576	0.0574	0.0581	0.0575

Table 3.6: Mean absolute time index, t_{abs} (in seconds), for the non-tuned (top row) and tuned (bottom row) MPC controllers.

Regarding the computational time required for control calculation, the tuned MPC controller is faster in both relative (t_{rel}) and absolute (t_{abs}) terms, with reductions of approximately 65% and 25%, respectively. This represents a marked improvement on both aspects, with the greater gain in t_{rel} attributable to the agent's tendency to select larger time step values dt. Such improvements translate into wider safety margins between the actual CPU time and the maximum time allowed between control updates.

Both tuned and non-tuned controllers exhibit consistent performance across all orbits and true anomalies, as shown in Figures 3.18, 3.19, 3.16, 3.17.

This behavior is expected, as the CPU time does not directly depend on the orbital dynamics, but only on the parameters dt and T_h. In the non-tuned MPC, t_{abs} and t_{rel} remain nearly constant, since dt and T_h are fixed. In contrast, the tuned controller can adjust dt and T_h, which results in slightly higher values of the time indexes under the most demanding conditions, at perigee in the most eccentric orbits.

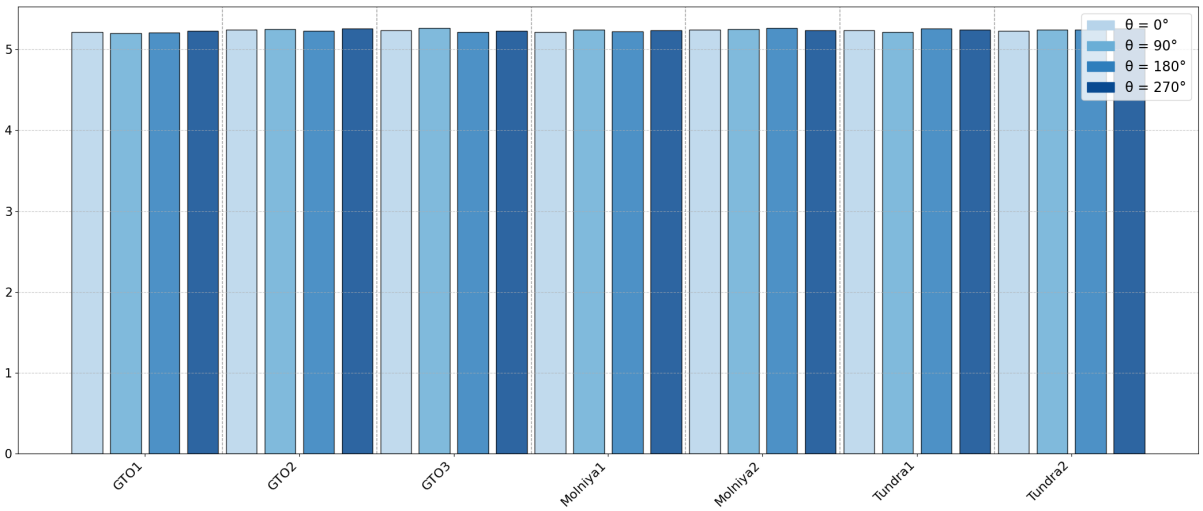

Figure 3.16: Mean values of the relative time index, t_{rel} (adimensional, percentage), non-tuned controller, broken down by orbit and true anomaly.

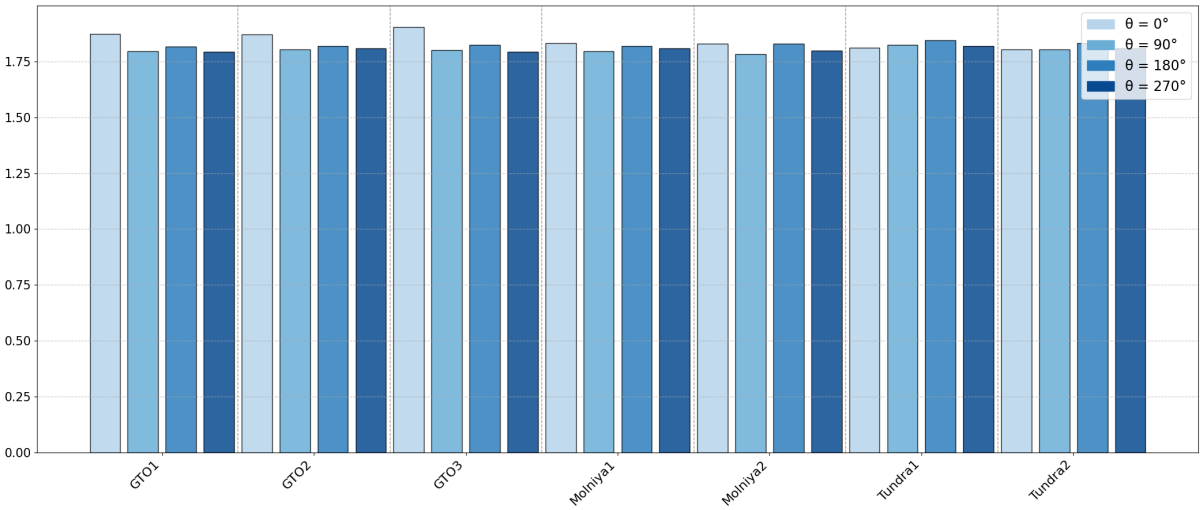

Figure 3.17: Mean values of the relative time index, t_{rel} (adimensional, percentage), tuned controller, broken down by orbit and true anomaly.

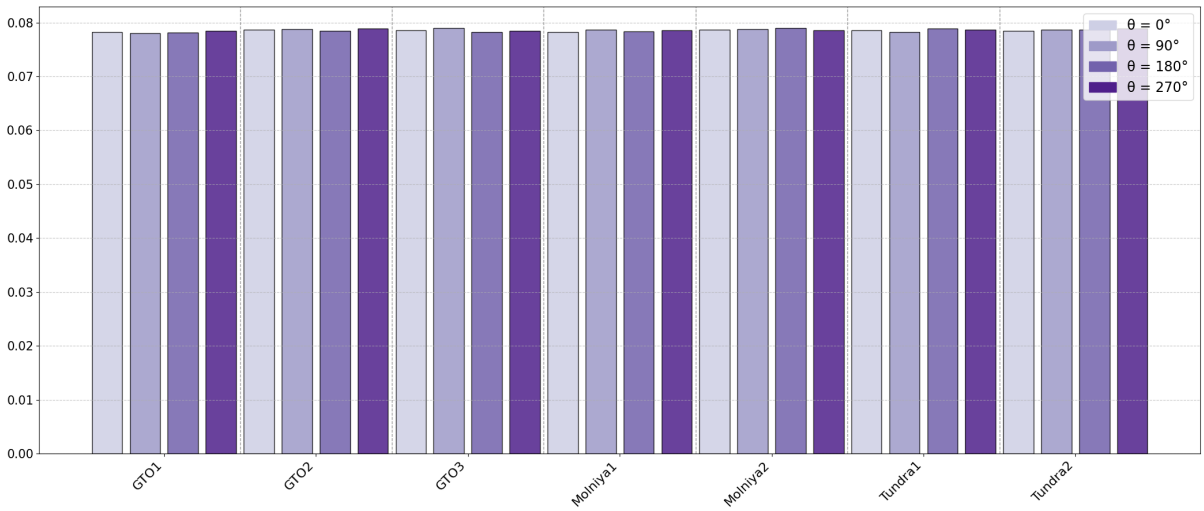

Figure 3.18: Mean values of the absolute time index, t_{abs} (in seconds), non-tuned controller, broken down by orbit and true anomaly.

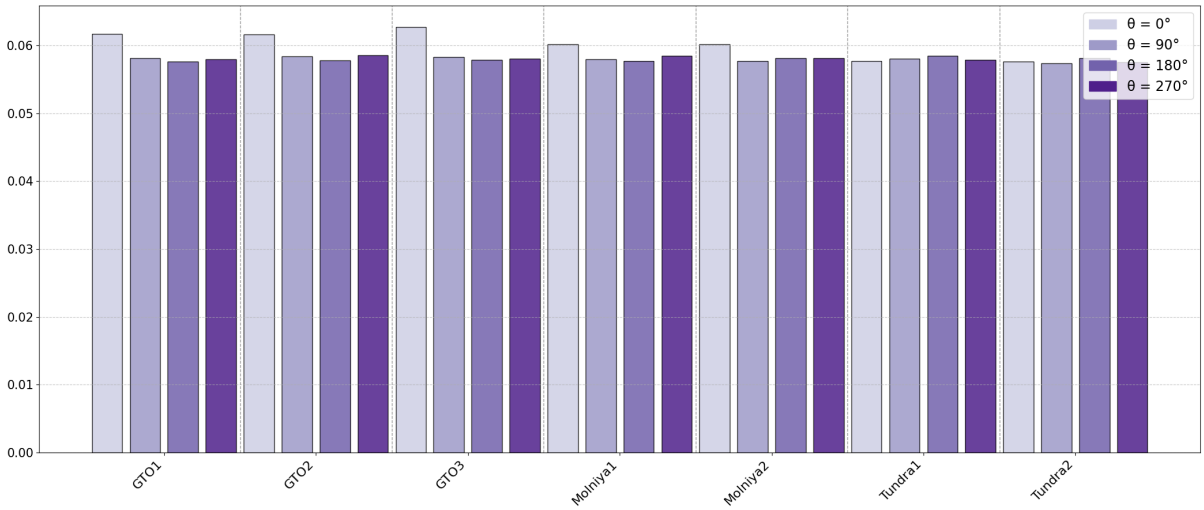

Figure 3.19: Mean values of the absolute time index, t_{abs} (in seconds), tuned controller, broken down by orbit and true anomaly.

4 | Conclusions and future developments

This thesis presented a novel hybrid control framework that integrates Deep Reinforcement Learning (DRL) with Model Predictive Control (MPC) for autonomous spacecraft proximity operations and docking in Highly Elliptical Orbits (HEO). The core contribution is a DRL agent trained to dynamically tune the key parameters of the MPC controller; specifically the prediction horizon T_h, control timestep Δt, and the state and control weighting matrices Q and R. This approach aims to enhance the adaptability and computational efficiency of MPC, leveraging its predictive, constraint-aware nature, while mitigating its high computational load and its reliance on manual tuning. This work demonstrates that such a DRL-tuned MPC is a viable and robust strategy for real-time, on-board implementation.
The key findings of this research and potential avenues for future exploration are summarized in this chapter.

4.1. Conclusions

The conclusions are structured as direct answers to the research questions that guided this work, first presented in the Introduction.

- **To what extent can Deep Reinforcement Learning be used to improve the computational efficiency and adaptability of Model Predictive Control for spacecraft proximity operations in HEO, without compromising the safety and performance guarantees traditionally associated with MPC?**

  This work provides definitive evidence that a DRL agent can successfully tune an MPC framework, yielding substantial improvements in both fuel efficiency and computational performance, all while consistently adhering to mission constraints. The DRL agent learns a sophisticated, state-dependent policy that maps the current state of the spacecraft to an optimal set of MPC parameters. This results in a sig-

nificant reduction in fuel consumption, with ΔV improvements ranging from 10% in highly perturbed orbital conditions to 40% in more favorable ones. Furthermore, the adaptability of the controller leads to a marked decrease in computational load, with absolute CPU time reduced by approximately 25% and relative time by 65%, thereby increasing the safety margins for real-time, on-board implementation.

Safety and performance guarantees are fundamentally preserved because the underlying MPC layer remains the final arbiter of control actions. The DRL agent acts as an expert tuner, but the MPC solver is still responsible for enforcing system dynamics and operational constraints (e.g. safety corridor, velocity limits) within its predictive model. The success of this hybrid system across extensive Monte Carlo campaigns validates its capacity to enhance performance without sacrificing the safety inherent to the MPC formulation.

1. **How does the choice of the MPC prediction horizon T_h and control timestep Δt affect the trade-off between computational load and control performance?**

   The investigation confirmed that a static choice for T_h and Δt is inherently suboptimal for a mission with dynamically varying phases. The key finding is that the DRL agent learns to manage this trade-off intelligently. It selects a shorter prediction horizon at the beginning of the maneuver—when the primary goal is to stabilize the chaser's velocity—and progressively extends it as the spacecraft approaches the target, where greater foresight is needed to navigate the narrow approach corridor safely. Concurrently, the agent adjusts the timestep Δt, often increasing it alongside T_h, to keep the number of optimization steps manageable. This dynamic tuning is the primary reason for the observed 65% reduction in relative computational time, increasing the safety margins for on-board implementation without degrading control quality.

2. **What is the effect of tuning the state and control weighting matrices Q and R on the stability, aggressiveness, and fuel efficiency of the resulting guidance behavior?**

   This research confirms that the real-time tuning of the Q and R matrices is the cornerstone of the controller's adaptability and a key driver of its enhanced performance. While a high Q/R ratio traditionally leads to more aggressive maneuvers and a low ratio favors fuel efficiency, the DRL agent moves beyond this simple trade-off. It learns a policy to continuously adjust these weights. For example, it prioritizes velocity error correction early on, then shifts focus

to fuel preservation during the mid-course approach, and finally prioritizes precision and safety in the terminal phase. This learned, multi-phase strategy enables the controller to achieve fuel savings of up to 40% compared to a fixed-parameter approach.

3. **Can DRL identify context-aware Q and R cost matrices that outperform static or heuristically selected ones, across a wide range of scenarios?**

   Yes. The DRL agent consistently identifies and applies context-aware Q and R matrices that lead to performance levels unattainable by any single fixed-parameter MPC baseline. The agent's ability to dynamically adjust its priorities—for instance, by decreasing the penalty on control effort (R) to allow for a necessary, large correction, and then increasing it during safer phases—is what drives the significant fuel savings and robust performance observed across all simulated orbital conditions. This state-dependent tuning represents a paradigm shift from traditional methods, which are unable to adapt their objectives with high level of granularity.

4. **How robust is the DRL-tuned MPC system to unmodeled disturbances and parameter uncertainties? How does it compare to a fixed MPC scheme?**

   The DRL-tuned MPC system demonstrated superior robustness, not only to random disturbances but also to the deterministic variations in orbital dynamics inherent to different HEOs. Monte Carlo simulations confirmed that the DRL-tuned system maintains a perfect mission success rate and lower average fuel consumption across a wide spectrum of off-nominal conditions, proving it to be a more reliable solution to a conventional fixed-parameter MPC.

   The DRL-tuned MPC system demonstrated superior robustness, not only to random disturbances but also to the deterministic variations in orbital dynamics inherent to different HEOs. This resilience is foundational to its design. By training the agent across all true anomalies of a demanding Geostationary Transfer Orbit (GTO), the system was systematically exposed to a wide spectrum of dynamics, including the highly perturbed and challenging conditions near perigee. Moreover, the training process explicitly incorporated navigation noise and thrust pointing errors, forcing the agent to develop a policy that is inherently robust to these real-world imperfections. Monte Carlo simulations confirmed that the DRL-tuned system maintains a perfect mission

success rate and lower average fuel consumption across a wide spectrum of off-nominal conditions, proving it to be a more reliable solution to a conventional fixed-parameter MPC.

4.2. Future Work

Building upon the foundations of this research, several promising directions for future work can be identified. These can be categorized into enhancements of the current framework and the exploration of alternative machine learning paradigms.

- **Enhancements to the DRL-Guided MPC Framework:**
  - **Expanded Action Space and Refined Reward Functions:** The current agent's action space could be extended to provide finer control, moving from diagonal scaling factors to the direct tuning of individual elements within the Q and R matrices. Furthermore, the reward function could be evolved into a multi-objective formulation to explicitly manage competing goals, such as minimizing fuel consumption versus minimizing time-to-dock.
  - **High-Fidelity 6-Degree-of-Freedom (6DOF) Dynamics:** To increase the technology readiness level of the system, the framework must be validated on a high-fidelity 6DOF simulation model. This would introduce the coupling between translational and rotational dynamics, requiring the DRL agent to learn a more complex policy that manages attitude control constraints, reaction wheel saturation, and avoids thruster plume impingement on the target.
  - **Exploration of Alternative DRL Paradigms:** While PPO proved effective, the framework's performance could be further enhanced by investigating state-of-the-art off-policy algorithms. Algorithms like Soft Actor-Critic (SAC) are particularly promising due to their high sample efficiency and robust exploration capabilities, driven by an entropy maximization framework. Such methods could potentially lead to faster convergence and discover more optimal tuning policies within the continuous, high-dimensional MPC parameter space.
  - **Time-Aware Reinforcement Learning Formulations:** Standard PPO relies on the theoretical assumption of a fixed duration for each environment step. In the present work, this assumption is violated, since the control timestep Δt is included in the agent's action space. As a consequence, an inconsistency arises: the discount factor γ is applied per decision step rather than per unit of time.

A promising direction for future work is the adoption of frameworks based on Semi-Markov Decision Processes (SMDPs), which employ a continuous-time discount factor (e.g., $\Gamma = \gamma \Delta t$). This approach ensures a temporally consistent valuation of rewards, thereby potentially providing a more stable training and superior control policy.

- **Alternative Machine Learning Solutions for Autonomous G&C:**
  - **Hierarchical Reinforcement Learning (HRL):** For a complete rendezvous and docking mission, an HRL architecture could be highly effective. A high-level agent selects strategic goals (e.g., "initiate far-range approach," "begin final alignment," "hold position"), while a lower-level agent, such as the one developed in this thesis, is responsible for tuning the MPC to achieve those sub-goals. This decomposition enhances learning efficiency and scalability for complex, multi-phase missions. [1, 3] This creates a powerful, multi-layered decision-making process that mirrors human mission planning and improves the interpretability of the autonomous agent's behavior. [1]
  - **Physics-Informed Neural Networks (PINNs) for Model Refinement:** Instead of tuning the controller, machine learning can be used to improve the model within the MPC. A PINN can be trained on-orbit to learn and correct for unmodeled dynamics or system degradation. By embedding the governing equations of motion into the network's loss function, a PINN ensures that its model corrections remain physically consistent, even with sparse on-orbit data. [4, 15] This provides the MPC with a more accurate prediction model, thereby improving its performance without changing the core tuning.
  - **Direct End-to-End Control vs. Hybrid Approach:** A valuable comparative study involves training an end-to-end DRL agent that outputs actuator commands directly and comparing its performance, safety, and interpretability with the DRL-guided MPC presented here. While end-to-end learning might discover a seemingly superior policy, its "black-box" nature complicates verification. The hybrid approach offers greater transparency and the inherent safety net of MPC's constraint handling, synergistically combining the adaptive intelligence of DRL with the verifiable, robust framework of model-based control. [16]

Bibliography

[1] M. Dentis, E. Capello, and G. Guglieri, "A novel concept for guidance and control of spacecraft orbital maneuvers," *International Journal of Aerospace Engineering*, vol. 2016, pp. 1–14, 2016.

[2] K. M. Nastasi, D. Thomas, K. Tetreault, I. Elliott, and J. Black, "Real-time optimal control, & tracking of autonomous micro-satellite proximity operations," in *AIAA SPACE 2016*, (Long Beach, California), AIAA, 2016.

[3] E. Capello, F. Dabbene, G. Guglieri, and E. Punta, "\"Flyable\" guidance and control algorithms for orbital rendezvous maneuver," *SICE Journal of Control, Measurement, and System Integration*, vol. 11, no. 1, pp. 14–24, 2018.

[4] G. Fereoli, H. Schaub, and P. Di Lizia, "Meta-reinforcement learning for spacecraft proximity operations guidance and control in cislunar space," *Journal of Spacecraft and Rockets*, vol. 62, no. 3, pp. 706–718, 2025.

[5] M. Mammarella, E. Capello, H. Park, G. Guglieri, and M. Romano, "Tube-based robust model predictive control for spacecraft proximity operations in the presence of persistent disturbance," *Aerospace Science and Technology*, vol. 77, pp. 585–594, 2018.

[6] W. J. Harris, *Visual Navigation and Control for Spacecraft Proximity Operations*. Thesis, Air Force Institute of Technology, 2021.

[7] A. Name, "Close-proximity operations around near-earth objects: Limitations of pid controllers," in *Lunar and Planetary Science Conference*, p. 2226, 2014.

[8] A. Name, "A quadratically-constrained lqr approach for finite-thrust orbital rendezvous with collision avoidance," *Journal of Guidance, Control, and Dynamics*, vol. 38, no. 4, pp. 700–712, 2015.

[9] A. Name, "H-infinity controller design for spacecraft terminal rendezvous on elliptic orbits using differential game theory," *Acta Astronautica*, vol. 127, pp. 123–134, 2016.

[10] M. Mancini, N. Bloise, E. Capello, and E. Punta, "Sliding mode control techniques

and artificial potential field for dynamic collision avoidance in rendezvous maneuvers." arXiv preprint arXiv:1906.10945, 2019.

[11] D. V. Andrea, L. Enrico, and D. Mantovani, "Autonomous rendezvous and docking maneuvers with model predictive control strategies." Manuscript.

[12] P. Razzaghi, A. Tabrizian, W. Guo, S. Chen, A. Taye, E. Thompson, A. Bregeon, A. Baheri, and P. Wei, "A survey on reinforcement learning in aviation applications," *Progress in Aerospace Sciences*, vol. 160, p. 102279, 2024.

[13] A. Brandonisio, M. Lavagna, A. M. A. Guardone, and P. Masarati, "Ai-based guidance for spacecraft proximity operations around uncooperative targets." Preprint.

[14] M. Tipaldi, R. Iervolino, and P. R. Massenio, "Reinforcement learning in spacecraft control applications: Advances, prospects, and challenges," *Annual Reviews in Control*, vol. 54, pp. 1–23, 2022.

[15] Y. Zhu, W. Cai, L. Yang, and H. Huang, "Flatness-based trajectory planning for electromagnetic spacecraft proximity operations in elliptical orbits," *Acta Astronautica*, vol. 152, pp. 342–351, 2018.

[16] S. Bai, "Exoatmospheric intercept guidance based on relative motion control," *Space: Science & Technology*, 2024.

[17] S. J. MacLellan, *Orbital Rendezvous Using an Augmented Lambert Guidance Algorithm*. PhD thesis, Massachusetts Institute of Technology, 2005. Thesis.

[18] A. Shirazi, J. Ceberio, and J. A. Lozano, "Optimal multi-impulse space rendezvous considering limited impulse using a discretized lambert problem combined with evolutionary algorithms," in *Proceedings of the 8th European Conference for Aeronautics and Space Sciences (EUCASS)*, 2019.

[19] M. Innocenti, "Dynamics and control analysis during rendezvous in non-circular orbits," *Frontiers in Space Technologies*, vol. 3, p. 929179, 2022.

[20] X. Wang, X. Yue, H. Wen, and J. Yuan, "Hybrid passive/active vibration control of a loosely connected spacecraft system," *Computer Modeling in Engineering & Sciences*, vol. 122, no. 1, pp. 61–87, 2020.

[21] *Guidance for Rendezvous and Formation Flying in Elliptical Orbits*. PhD thesis, Delft University of Technology, 2023. Dissertation.

[22] M. Leomanni, E. Rogers, and S. B. Gabriel, "Explicit model predictive control ap-

proach for low-thrust spacecraft proximity operations," *Journal of Guidance, Control, and Dynamics*, vol. 37, no. 6, pp. 1780–1790, 2014.

[23] J. J. Kaczmarek and C. Zagaris, "Autonomous multi-phase rendezvous, proximity operations, and docking via model predictive control," in *2023 IEEE Aerospace Conference*, (Big Sky, MT, USA), pp. 1–10, IEEE, 2023.

[24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2 ed., 2018.

[25] M. Ławryńczuk, *Introduction to Model Predictive Control*. Springer, 2020.

[26] D. P. Bertsekas, "Model predictive control and reinforcement learning: A foundational connection," in *IFAC Workshop on Nonlinear Model Predictive Control*, 2024.

[27] E. Bøhn, S. Gros, S. Moe, and T. A. Johansen, "Optimization of the model predictive control meta-parameters through reinforcement learning," *Engineering Applications of Artificial Intelligence*, vol. 123, p. 106211, 2023.

[28] C. F. O. d. Silva, A. Dabiri, and B. De Schutter, "Integrating reinforcement learning and model predictive control with applications to microgrids," *arXiv preprint arXiv:2409.11267*, 2024.

[29] E. Bøhn, S. Gros, S. Moe, and T. A. Johansen, "Reinforcement learning of the prediction horizon in model predictive control," *arXiv preprint arXiv:2102.11122*, 2021.

[30] M. E. Bagle and F. Goia, "Combined reinforcement learning (rl) and model predictive control (mpc) for optimal building energy use," in *Proceedings of the 2023 International Building Performance Simulation Association Conference*, 2023.

[31] H. D. Curtis, *Orbital Mechanics for Engineering Students*. Oxford, UK: Butterworth-Heinemann, 2014.

[32] V. Pesce, A. Colagrossi, and S. Silvestrini, *Modern Spacecraft Guidance, Navigation, and Control: From System Modeling to AI and Innovative Applications*. Amsterdam: Elsevier, 2023.

[33] J. T. Emmert, W. K. Tobiska, D. R. Weimer, and J. Yoshii, "Nrlmsis 2.0: A whole-atmosphere empirical model of temperature and composition," 2021.

[34] E. F. Camacho and C. Bordons, *Model Predictive Control*. London, UK: Springer, 2nd ed., 2004.

[35] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Madison, WI, USA: Nob Hill Publishing, 2nd ed., 2017.

[36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[37] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[38] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[39] D. Pinard *et al.*, "Accurate and autonomous navigation for the atv," *Acta Astronautica*, 2007. Techniques for precise autonomous navigation of ESA's Automated Transfer Vehicle.

[40] N. Escudero and S. R. Curtis, "Rendezvous proximity operations strategies to the iss," in *AIAA SciTech Forum*, 2025. Strategies for rendezvous and proximity operations with Cygnus and Dragon vehicles at the ISS.

[41] G. Borelli *et al.*, "Safe guidance scheme for proximity operations forced motion," *Journal of Guidance, Control, and Dynamics*, 2025.

[42] "International docking system standard (idss), interface definition document (idd), revision f," tech. rep., International Docking System Standard Committee (NASA, ESA, JAXA, Roscosmos, CSA), July 2022. Approved for Public Release.

[43] P. S. Maybeck, *Stochastic Models, Estimation, and Control*, vol. 1. New York, NY, USA: Academic Press, 1979.

[44] P. Fortescue, G. Stark, and J. Swinerd, *Spacecraft Systems Engineering*. Chichester, UK: Wiley, 4th ed., 2011.

[45] A. Kosari, A. Badamchizadeh, and F. Gharaveisi, "An optimal fuzzy pid control approach for docking maneuver of spacecraft," *Journal of Space Safety Engineering*, 2017. Proposes an optimal fuzzy-PID control strategy for spacecraft docking maneuvers, analyzing performance and stability under orbital disturbances and constraints.

[46] S. D. Cairano, H. Park, and I. Kolmanovsky, "Model predictive control approach for guidance of spacecraft rendezvous and proximity maneuvering," *International Journal of Robust and Nonlinear Control*, vol. 22, no. 12, pp. 1398–1427, 2012.

[47] A. Brandonisio, L. Capra, and M. Lavagna, "Network architecture and action space analysis for deep reinforcement learning towards spacecraft autonomous guidance," *Advances in Space Research*, vol. 71, pp. 3787–3802, May 2023.

List of Figures

1 Schematics of a typical conic approach corridor 2
2 Geostationary Transfer Orbit (GTO) and Molniya orbit trajectories. 4

1.1 Coordinate frames in relative motion. 10
1.2 Diagram of an LSTM unit showing the flow of data through the cell and gates. 22

2.1 Representation of the approach corridor in the X-Y plane, LVLH frame. . . 27
2.2 Pyramidal constraint built into the linear MPC, fully inscribed in the conical approach region. Relative LVLH frame. 30
2.3 Classical MPC weights over time, showing increase of Q_p as the chaser approaches the target 32

3.1 Example of a RVD trajectory with classical MPC controller, LVLH frame . 45
3.2 Chaser relative velocity and position, LVLH frame 45
3.3 Simulated trajectories on the reference GTO orbit, classical (non-tuned) MPC, LVLH frame 46
3.4 Histograms of the main performance indexes on the reference orbit, classical (non-tuned) MPC 47
3.5 Mean rollout reward during the training phase for CPU cost 48
3.6 Histograms of the main performance indexes on the reference orbit, RL-tuned MPC on dt and T_h 49
3.7 Time step and Prediction horizon provided by the agent during the RVD episode 50
3.8 Mean rollout reward during the training for Fuel efficiency and Constraint satisfaction 51
3.9 Simulated trajectories on the reference GTO orbit, RL-tuned MPC, LVLH frame 51
3.10 Histograms of the main performance indexes on the reference orbit, RL-tuned MPC on Q and R 52

3.11 Q_p, Q_v and R values during the RVD episode, on the reference orbit, after training . . . 53
3.12 Mean values of ΔV [m/s] for the non-tuned controller, broken down by orbit and true anomaly. . . . 57
3.13 Mean values of ΔV [m/s] for the tuned controller, broken down by orbit and true anomaly. . . . 57
3.14 Mean values of constraint index (dimensionless) for the non-tuned controller, broken down by orbit and true anomaly. . . . 59
3.15 Mean values of constraint index (dimensionless) for the tuned controller, broken down by orbit and true anomaly. . . . 59
3.16 Mean values of the relative time index, t_{rel} (adimensional, percentage), non-tuned controller, broken down by orbit and true anomaly. . . . 62
3.17 Mean values of the relative time index, t_{rel} (adimensional, percentage), tuned controller, broken down by orbit and true anomaly. . . . 62
3.18 Mean values of the absolute time index, t_{abs} (in seconds), non-tuned controller, broken down by orbit and true anomaly. . . . 63
3.19 Mean values of the absolute time index, t_{abs} (in seconds), tuned controller, broken down by orbit and true anomaly. . . . 63

List of Tables

2.1 Orbital parameters of the GTO used for training. 26
2.2 Chaser spacecraft physical parameters. 26
2.3 Docking tolerances from International Docking System Standard [42] . . . 27
2.4 Architecture of the Actor (A) and Critic (C) networks used in the Recurrent PPO agent. 37
2.5 Hyperparameters used for training the Recurrent PPO agent. 38
2.6 Architecture of the Actor (A) and Critic (C) networks used in the Recurrent PPO agent. 42
2.7 Hyperparameters used for training the Recurrent PPO agent. 42

3.1 Orbital parameters of the verification orbits used for performance assessment. 54
3.2 Mean ΔV values for non-tuned (top row) and tuned (bottom row, color-coded) MPC controllers. 55
3.3 Mean ΔV values for non-tuned (top row) and tuned (bottom row, color-coded) MPC controllers. 56
3.4 Mean constraint index values for non-tuned (top row) and tuned (bottom row) MPC controllers. Color-coding for the tuned controller indicates performance relative to the non-tuned one: green for better, yellow for slightly worse, and orange for more than 2 times worse. 58
3.5 Mean relative time index, t_{rel} (adimensional, percentage), for the non-tuned (top row) and tuned (bottom row) MPC controllers. 60
3.6 Mean absolute time index, t_{abs} (in seconds), for the non-tuned (top row) and tuned (bottom row) MPC controllers. 61